Query Campus: A Retrieval-Augmented Generation Based Conversational Assistant For College Website Information Access

A Report submitted
in partial fulfillment for the degree of

Bachelor of Engineering

in

Information Technology

(Session 2026-2027)

Submitted By:
Akshat Sharma (23BIT005)
Tanmay Choudhary (23BCS072)
Riya Rawat (23BIT060)

Guided By:
Vedpriya Dongre

Department of Information Technology
Institute of Engineering & Technology
Vikramshila Campus
Devi Ahilya Vishwavidyalaya, Indore (M.P.)
(https://ietdavv.edu.in)

October 2026

CERTIFICATE

The project report entitled “Query Campus: A Retrieval-Augmented Generation Based Conversational Assistant For College Website Information Access” submitted by Akshat Sharma, Tanmay Choudhary, Riya Rawat is approved for partial fulfillment for the award of Bachelor of Engineering in Information Technology degree by Devi Ahilya Vishwavidyalaya, Indore.

Internal Examiner External Examiner

Recommendation

The dissertation entitled “Query Campus: A Retrieval-Augmented Generation Based Conversational Assistant For College Website Information Access” submitted by Akshat Sharma, Tanmay Choudhary, Riya Rawat is a satisfactory account of the bonafide work done under my supervision and is recommended towards the partial fulfillment for the award of Bachelor of Engineering in Information Technology degree by Devi Ahilya Vishwavidyalaya, Indore.

(Signature of Project Guide)

Date:

VEDPRIYA DONGRE
[Designation of Project Guide]
Department of Computer Engineering

(Signature of HOD)

Counter signature of HOD with seal

Candidate Declaration

We hereby declare that the work which is being presented in this project entitled "Query Campus: A Retrieval-Augmented Generation Based Conversational Assistant For College Website Information Access" in partial fulfillment of degree of Bachelor of Engineering in Information Technology is an authentic record of our own work carried out under the supervision and guidance of Vedpriya Dongre, [Designation of Project Guide] in Department of Computer Engineering, Institute of Engineering and Technology, Devi Ahilya Vishwavidyalaya, Indore.

We are fully responsible for the matter embodied in this project in case of any discrepancy found in the project and the project has not been submitted for the award of any other degree.

Date:
Place: Indore

(Signature of student 1)
Akshat Sharma
23BIT005

(Signature of student 2)
Tanmay Choudhary
23BCS072

(Signature of student 3)
Riya Rawat
23BIT060

SELF-DECLARATION OF PLAGIARISM REPORT

The project report entitled “Query Campus: A Retrieval-Augmented Generation Based Conversational Assistant For College Website Information Access” submitted by Akshat Sharma, Tanmay Choudhary, Riya Rawat in partial fulfillment of the requirement for the award of the degree of B.E. in Information Technology has been checked for plagiarism using a standard plagiarism detection tool.

The similarity index of the report is _____ %, which is within the permissible limit as per institute/university norms.

We hereby declare that:

- The work submitted is original and has not been copied from any source without proper citation.
- All referenced materials have been duly acknowledged.
- In case of any discrepancy found in future, we shall be solely responsible for the same.

Signature of Student(s):

1. Akshat Sharma (23BIT005) _____
2. Tanmay Choudhary (23BCS072) _____
3. Riya Rawat (23BIT060) _____

Date: _____

ACKNOWLEDGEMENTS

We express our deepest gratitude to our project guide, Vedpriya Dongre, Department of Computer Engineering, for her constant guidance, patient supervision and encouragement throughout the development of this project. She helped us to shape a broad idea about making the college website easier to use into a well-defined problem, reviewed our synopsis and requirement specification, and challenged us to measure the accuracy of the system rather than only demonstrate it. Her suggestions during the review meetings directly influenced the retrieval design and the evaluation approach used in this report.

We are thankful to [Name of HOD], Head of the Department, for providing the departmental facilities, a supportive academic environment and the permission to carry out this work, and for his timely advice on the scope of the project.

We are grateful to [Name of Director], Director, Institute of Engineering & Technology, Devi Ahilya Vishwavidyalaya, Indore, for providing the institutional infrastructure, computing facilities and internet access without which the crawling, indexing and testing of the system would not have been possible. The publicly available information on the institute’s website and its class timetable portal formed the knowledge base of this project.

We also thank all the faculty members of the department for their valuable suggestions, our friends and classmates who tested the application and shared honest feedback on the questions they would like to ask a campus assistant, and our families for their continuous support and motivation.

ABSTRACT

Information about a college is spread over many web pages, PDF notices and a separate timetable portal. Students and applicants lose a lot of time searching for admission rules, fees, teacher details and class timetables. General chatbots often answer such college questions with wrong facts. This project, Query Campus, is a chat assistant that answers questions about the Institute of Engineering & Technology, DAVV, Indore in simple English. It uses only the information published by the institute and shows the pages it used.

The system uses Retrieval-Augmented Generation. A crawler and a timetable scraper collect 1,009 documents, which are cut into 2,864 small chunks. Each chunk is converted into numbers with Voyage AI and stored in a Pinecone database. For every question, the backend searches by meaning and by keywords, merges the results, and sends the best 16 chunks to Google Gemini. Gemini is told to answer only from these chunks. If one of the two searches is not available, the other one still gives an answer. The answer is streamed to a React chat screen with a calm, light design, together with source links and downloadable documents. Retries, backup models, time limits and a limit on questions per visitor keep the website safe to run for the public.

On a test set of 18 real questions, the assistant answered all correctly, usually in 1.5 to 4 seconds, and it refused an unanswerable question instead of guessing. The project shows that a small, low-cost and self-hosted system can give a college a reliable question-answering service.

TABLE OF CONTENTS

DESCRIPTION **Page No**

Certificate..... i

Recommendation..... ii

Candidate Declaration..... iii

Plagiarism Check Report..... iv

Acknowledgements..... v

Abstract..... vi

Chapter 1 Introduction..... **1**

1.1 Overview and Issues Involved..... 1

1.2 Problem Definition..... 2

1.3 Proposed Solution..... 3

Chapter 2 Literature Survey..... **5**

2.1 Existing Solutions..... 5

2.2 Methodology..... 7

Chapter 3 Analysis & Design..... **10**

3.1 Software Requirements..... 10

3.2 Hardware Requirements..... 11

3.3 Analysis Diagrams..... 12

3.4 Design Diagrams..... 24

Chapter 4 Implementation and Testing..... **30**

4.1 Database Design..... 30

4.2 Class Diagram..... 35

4.3 Test Cases..... 44

Chapter 5 Conclusion..... **54**

References..... **56**

Appendix..... **58**

Chapter-1 Introduction

1.1 Overview and Issues Involved

A college website is the first place where students and applicants look for information. They want to know about admission, fees, scholarships, exam notices, syllabus and class timetables. In reality, this information is spread over many pages and files. The IET DAVV website (Institute of Engineering & Technology, Devi Ahilya Vishwavidyalaya, Indore) gives our project 160 web pages and 606 PDF files. The current class timetables are on a separate portal (timetable.ietonline.in), and we turn them into 243 more documents, one for each class and each teacher. If a student only wants to know who the class coordinator of fourth year CSE section B is, they first have to guess which page or PDF has the answer.

AI chatbots such as Google Gemini can answer questions in simple language. But they have not read the college notices, and they often give wrong answers in a confident tone. This is called hallucination. A wrong fee or date can cost a student time and money. Retrieval-Augmented Generation (RAG) reduces this problem. The system first finds the right text from the college website and then asks the AI to answer using only that text.

Building such an assistant for a real college has several issues:

- **Scattered content.** Information is inside web pages full of menus and footers, and inside PDF files. The useful text has to be separated from the rest.
- **More than one source.** Timetables are on another site and are shown as grids. They must be changed into text that can be searched by class, subject and teacher name.
- **Search quality.** Search by meaning is good for topics, but weak for exact names, course codes and numbers. One kind of search is not enough for a question like “What does Rohit Pathak teach?”.
- **Trust.** Answers must come only from the college text, must show their sources, and must say “I don’t know” when the answer is not there.
- **Fresh data.** Notices and timetables change every semester, so the data must be easy to refresh.

- **Free limits.** The project uses free plans of Voyage AI, Pinecone and Gemini. These allow only a few requests per minute, so the system needs waiting, retrying and backup models.
- **Easy to use.** Students expect a chat screen where answers appear quickly, old chats are saved, and documents like a timetable can be opened in one click.

Query Campus solves these issues with a step-by-step pipeline, which is explained in the next chapters.

1.2 Problem Definition

Important information about IET DAVV is available only as many web pages, PDF files and a separate timetable portal. There is no single place where a student can ask a question in simple language and get a correct answer with its source. General AI chatbots are not reliable for this, because they do not have the college data and may make up answers.

So the problem is to build a web-based chat assistant that collects and stores the college information, finds the most useful parts for a question, and writes a short answer using only those parts. It should show the sources, and it should refuse to guess when the information is missing. The objectives are:

1. To collect text from the college website (web pages and PDFs) and from the timetable portal.
2. To cut the text into small pieces, convert each piece into numbers (embeddings), and store them in a vector database with their source.
3. To find relevant pieces using both meaning-based search and keyword search.
4. To generate an answer with Gemini that is based on the found text, has citations, and appears word by word.
5. To show the answer in a simple, clean chat screen with saved chats, source links and document cards.
6. To measure the accuracy of the system with a fixed set of test questions.

1.3 Proposed Solution

Query Campus is a RAG-based chat assistant. It has a NestJS backend and a React frontend in one code repository. It works in two parts, shown in the flow diagrams of Fig. 1.1. The first part is **indexing**. The operator runs it whenever the website changes. The crawler and the timetable scraper collect documents, which are cut into chunks, converted to embeddings and saved in Pinecone. The second part is **answering**. For each question, the backend searches by meaning and by keywords at the same time, merges the two result lists, and sends the best 16 chunks to Gemini. Gemini writes the answer, and the answer is streamed to the browser. Citation numbers like [2] are changed into links. If the user asked for a document, the document is shown as a card.

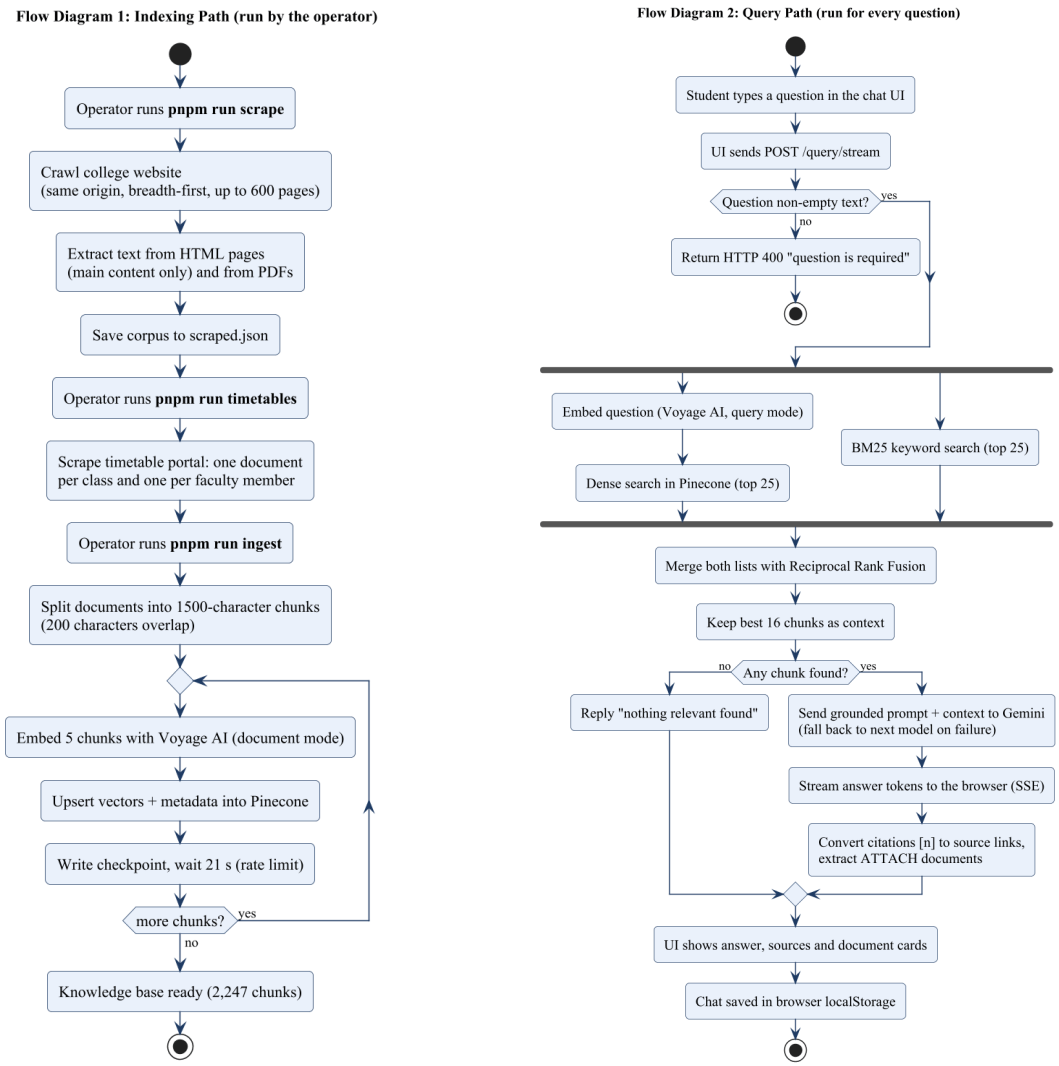


Fig. 1.1: Flow diagrams of Query Campus (left: indexing, right: answering)

Functions developed. The following functions are built and tested:

- Crawling of the college website (up to 600 pages, four at a time) and text extraction from web pages and PDFs.
- Scraping of the timetable portal into one text document per class and per teacher, with subjects, rooms, coordinator and weekly slots.
- Chunking (1,500 characters with 200 overlap), embedding with Voyage AI (voyage-3, 1024 numbers per chunk) and saving in a Pinecone index. A stopped run can continue from where it stopped.
- Search by meaning plus keyword search (BM25), merged with Reciprocal Rank Fusion.
- Answer generation with Gemini that uses only the found text, shows citations, says “I don’t know” when needed, and switches to another model if one fails.
- A chat screen with a light, calm theme, saved chats, source links and document cards with Open and Download buttons.
- A safe download route that allows only PDF files from the college website, and a health check route.
- Safe running for the public: a limit on questions per visitor, a list of allowed website addresses (CORS) and a maximum question length.
- Fast updating: the incremental mode adds only new pages to the index, so a new crawl takes minutes instead of hours.
- A test tool with 18 questions that checks both search and answers and compares with the last run.

Why it is better than existing solutions. The college website search cannot understand questions. FAQ chatbots only answer fixed questions. General chatbots make up facts. Paid college chatbot products are closed and are not made for the IET DAVV site and timetable portal. Query Campus is different in six ways: (1) every answer comes from college pages and shows its sources; (2) keyword search helps with names and codes; (3) it also reads the timetable portal and can show the timetable document; (4) it refuses to answer instead of guessing; (5) it is self-hosted, uses free plans and is updated with two commands; (6) its accuracy is measured with a test set.

Chapter-2 Literature Survey

2.1 Existing Solutions

This section describes the existing ways of finding information on a college website, with their good and bad points. At the end, we explain the gap that this project fills. Table 2.1 compares them.

(a) The IET DAVV website. The official website (ietdavv.edu.in) has information on admissions, departments, faculty, syllabus, notices and downloads. It is the most trusted source. But related facts are in different menus, many facts are only inside PDF files, and a user must already know where to look. A question cannot be asked directly.

(b) The class timetable portal. The timetable portal (timetable.ietonline.in) is linked from the college home page. The user selects a session, semester and course, and the portal shows the timetables. It is accurate and current, but it only answers “show me this class”. To find which classes a teacher takes, the user must open many timetables.

(c) Perplexity AI. Perplexity [1] is a chat-style search engine. It finds web pages and writes an answer with links. It shows that retrieval plus generation reduces wrong answers. But it searches the whole web, so the answer to a college question may come from an unofficial or old page.

(d) ChatPDF. ChatPDF [2] lets a user upload a PDF and ask questions about it. It is useful for one document. But the college has more than 400 PDFs, and the user would still have to find and upload the right one.

(e) Paid chatbots for colleges. Ivy.ai [3], Ocelot [4], Element451 BoltBot [5] and Gravyty [6] sell AI chatbots to universities. They show that such chatbots work in real colleges. But they are closed, paid products set up by the vendor, and a single college with a small budget cannot easily self-host them.

(f) General AI chatbots. Tools like ChatGPT or Gemini give fluent answers, but they have not read the current college notices. They can give plausible but wrong answers, which is not acceptable for fees, dates or eligibility.

(g) Rule-based FAQ chatbots. These chatbots answer only a fixed list of questions. They fail on anything else, and someone must edit them by hand whenever a notice changes.

(h) Background ideas. RAG gives a text generator some retrieved passages to work with. Search by meaning can do better than plain keyword search for open questions, while BM25 keyword ranking is still very good for exact words. Reciprocal Rank Fusion is a simple way to merge result lists from different searches. Query Campus uses all three ideas together on the data of one college.

Table 2.1: Comparison of existing solutions with Query Campus

Solution	What it does	Advantages	Drawbacks	Gap that Query Campus fills
IET DAVV website and its menus / search	Pages, PDF notices, downloads, faculty lists	Official and complete	Information is scattered; keyword search only; no direct answers	Direct answers in simple language, with sources
Timetable portal (timetable.ietonline.in)	Shows class timetables after choosing session, semester and course	Accurate and current	User must know the class; cannot ask who teaches what	Questions about teachers, subjects and coordinators; timetable shown in chat
Perplexity AI [1]	Web search with a chat answer and links	Answers with citations	Searches the whole web; may use unofficial pages	Uses only official IET DAVV sources
ChatPDF [2]	Questions on one uploaded PDF	Simple document Q&A	One file at a time; user must upload it	Whole site and portal are ready in advance
Ivy.ai / Ocelot / Element451 BoltBot / Gravyty [3]-[6]	Paid chatbots for colleges	Ready to use, connect with college systems	Closed, paid, need vendor setup; not made for IET DAVV	Open, self-hosted, works on free plans

Solution	What it does	Advantages	Drawbacks	Gap that Query Campus fills
General AI chatbots (ChatGPT, Gemini)	Open conversation	Fluent and flexible	Do not have college data; may make up facts; no sources	Answers only from college text, with sources and refusal
Rule-based FAQ chatbots	Fixed menus and ready replies	Predictable	Fail on new questions; need manual updates	Free questions; easy re-indexing

Gaps and why this project is needed. None of the above gives a student a direct answer that comes only from the official IET DAVV website and timetable portal, names the person or document that answers it, and can be refreshed cheaply when the content changes. Website search and timetable filters need the user to know where to look. FAQ bots are too fixed. General chatbots cannot be trusted. Paid products are closed. Query Campus fills this gap with an open, self-hosted RAG system made for IET DAVV, and its accuracy is tested.

2.2 Methodology

2.2.1 Steps followed during development

The project was built step by step. Each step gave a working part that could be tested before the next part was added.

1. **Requirements.** The synopsis and the SRS fixed the scope: answer questions about the college website with sources, and refuse when the answer is not available.
2. **Collecting data.** A crawler was built with Axios and Cheerio, with pdf-parse for PDF text. A separate scraper was added for the timetable portal.
3. **Indexing.** A chunker, an embeddings client (Voyage AI) and a vector database client (Pinecone) were joined into the ingestion service. Batching, retries and checkpoints were added because of the free-plan limits.
4. **Search and answers.** First, search by meaning and a Gemini prompt were built. After testing showed that names and numbers were missed, BM25 keyword search and Reciprocal Rank Fusion were added.

5. **Reliability.** Retry with waiting, backup models, a time limit for the first word, and a cache for question embeddings were added.
6. **User interface.** A React chat app was built with streaming answers, markdown, saved chats, source links and document cards. Later, the design was changed to a simple, light theme.
7. **Testing.** A set of 18 real questions with expected facts was written and run after each change. API checks, unit tests and end-to-end tests were added. Problems found in testing were fixed (see 4.3.4).
8. **Documents.** A README, the SRS and this report explain the system.

2.2.2 Concepts needed to understand the project

Web crawling. A crawler opens a page, collects its links and repeats this for every new link. Breadth-first means it visits page by page, level by level. A list of visited pages avoids loops. Allowing only links of the same site keeps the crawl inside the college website.

HTML parsing and PDF text extraction. Cheerio can remove menus, scripts and footers and pick the main content of a page. For PDFs, the text layer is read. Scanned PDFs that are only pictures have no text layer and cannot be read without OCR.

Chunking. AI models handle limited text, and a search result should be a short, focused part. So each document is cut into chunks. A small overlap between chunks keeps sentences that fall on a border.

Embeddings and cosine similarity. An embedding model converts text into a list of numbers (a vector). Texts with similar meaning get similar vectors. The similarity of two vectors a and b is measured by $\cos(a, b) = (a \cdot b) / (|a| \, |b|)$. Voyage AI uses one mode for stored documents and another for questions.

Vector database. A vector database stores vectors with extra data and quickly finds the vectors closest to a question vector. Pinecone is a managed service for this. Our index uses cosine distance and 1024 numbers per vector.

BM25 keyword search. BM25 gives a score to each text based on how often the query words appear in it. It is very good at exact words such as names, codes and numbers.

Reciprocal Rank Fusion (RRF). RRF merges several ranked lists. Each item gets 1 / (k + rank) from every list, where k is 60 here, and the scores are added. An item that is high in any list moves up, and the scores of different searches need not be compared.

Large language model and grounding. A large language model (LLM) is a neural network that writes text. Grounding means telling the model to use only the given text, to cite it, and to say when the answer is missing. This is the main protection against hallucination.

Retrieval-Augmented Generation. RAG means: find passages for the question, put them in the prompt, and let the model write the answer. To update the knowledge, we only re-index. The model is not retrained.

REST and server-sent events. The backend offers REST routes over HTTP with JSON. For streaming, the server keeps one response open and sends small events like “event: delta” with the new text. The browser reads them one by one, so the user sees the answer while it is being written.

Handling failures. Retrying after a wait, honouring the Retry-After header, a time limit on the first word, and switching to another model keep the service working when an outside API is slow or limited.

Measuring quality. Search is checked by looking at whether the page that holds the answer is among the chunks sent to the model, and at its rank. Answers are checked for the required facts, and for attachments that appear only when requested.

Chapter-3 Analysis & Design

3.1 Software Requirements

The software below was chosen because it is reliable, has a free plan and lets us use one language (TypeScript) everywhere. The reason for each choice is given in Table 3.1. The project also needs three online services (Voyage AI, Pinecone and Google Gemini). Their API keys are kept in the environment file. Public access to the college website and the timetable portal is also needed.

Table 3.1: Software requirements, technologies and tools

Layer	Technology (version)	Purpose and reason for choice
Runtime	Node.js (LTS, ES modules; developed on v24)	One language (TypeScript) for backend, scripts and frontend; non-blocking I/O suits many concurrent network calls to third-party APIs and streaming responses.
Backend framework	NestJS 12, Express platform	Modular architecture with dependency injection; each concern (scraper, ingestion, embeddings, vector store, RAG) is a separate testable module.
Language	TypeScript 6	Static typing catches errors in data contracts such as ScrapedDocument and Chunk.
HTTP client	Axios 1.x	Streaming responses, time-outs, abort signals and typed errors for retry decisions.
HTML parsing	Cheerio 1.x	Fast server-side jQuery-like parsing to remove page chrome and select the main content.
PDF extraction	pdf-parse 2.x	Extracts the text layer of notices, forms and timetables published as PDF.
Keyword search	MiniSearch 7.x	In-process BM25 index with fuzzy and prefix matching; complements semantic search for names and codes.
Concurrency	p-limit 7.x	Bounds the crawler to four simultaneous requests to stay polite to the college server.
Embeddings	Voyage AI voyage-3 (1024 dimensions)	Managed embedding API with separate document and query modes and good retrieval quality; no GPU needed.

Layer	Technology (version)	Purpose and reason for choice
Vector database	Pinecone serverless (AWS us-east-1, cosine)	Managed approximate nearest-neighbour search with metadata; no self-hosting; free tier sufficient.
Generation	Google Gemini (default gemini-3.5-flash-lite with two fall-back models)	Low latency and cost, large context window, streaming API; fall-back models improve availability.
Frontend	React 19, Vite 8, Tailwind CSS 4, react-markdown, motion	Component-based UI, fast build, utility-first styling, markdown answers and animations.
Testing and quality	Vitest 4, Supertest, oxlint, Prettier	Unit and end-to-end tests, linting and formatting in the same TypeScript toolchain.
Package management	pnpm workspaces	One repository for frontend and backend with efficient dependency sharing.
Configuration	.env files, @nestjs/config	API keys are kept out of source control and out of the browser.

Main requirements in brief. The system must collect text; cut, embed and store it; take a question and find, write and stream an answer with sources; give requested documents; keep chat history in the browser; and report its health. **Other requirements:** answers within a few seconds, safe behaviour when a service is slow or limited (retry and backup model), no secret keys in the browser, a download route that cannot be misused, a screen that works on phones, and a repeatable accuracy test.

3.2 Hardware Requirements

Query Campus needs no special hardware and no GPU, because the heavy work (embedding and answer writing) is done by online services. So the needs in Table 3.2 are small. The reason for each is given in the table.

Table 3.2: Hardware requirements

Item	Minimum requirement	Reason
Processor	Dual-core 2 GHz (any x86-64 or ARM64)	All heavy computation (embedding, generation) is done by remote APIs; the server only parses text, fuses rankings and relays streams.
Memory	4 GB (8 GB recommended for development)	The BM25 index of about 2,250 chunks and the Node.js process need a few hundred MB; the crawler holds pages in memory; the IDE and browser need the rest.
Storage	1 GB free	Corpus JSON, keyword cache (about 3 MB), eval caches and node_modules.
Network	Broadband, outbound HTTPS to Voyage, Pinecone, Gemini and the college sites	Every question calls three external services; crawling downloads hundreds of pages and PDFs.
Display and input	Any device with a modern browser (Chrome, Edge, Firefox, Safari)	The client is a responsive single-page application usable down to 360 px width.
Server (deployment)	1 vCPU, 1 GB RAM virtual machine or container	Sufficient for a single-institution workload because the service is stateless.

3.3 Analysis Diagrams

3.3.1 Use Case Model

Fig. 3.1 shows the actors and what they can do. The main actor is the **user** (student, applicant or staff), who asks questions, reads answers, downloads documents and manages chats. The **operator** (a team member or administrator) keeps the data up to date and checks quality. The other actors are outside systems: the college website and timetable portal (data sources), Voyage AI (embeddings), Pinecone (storage and search) and Google Gemini (answers). “Ask a Question” always uses “Retrieve Relevant Chunks” and “Generate Grounded Answer”.

Viewing sources and downloading documents are extra steps that happen only when the answer has sources or attachments.

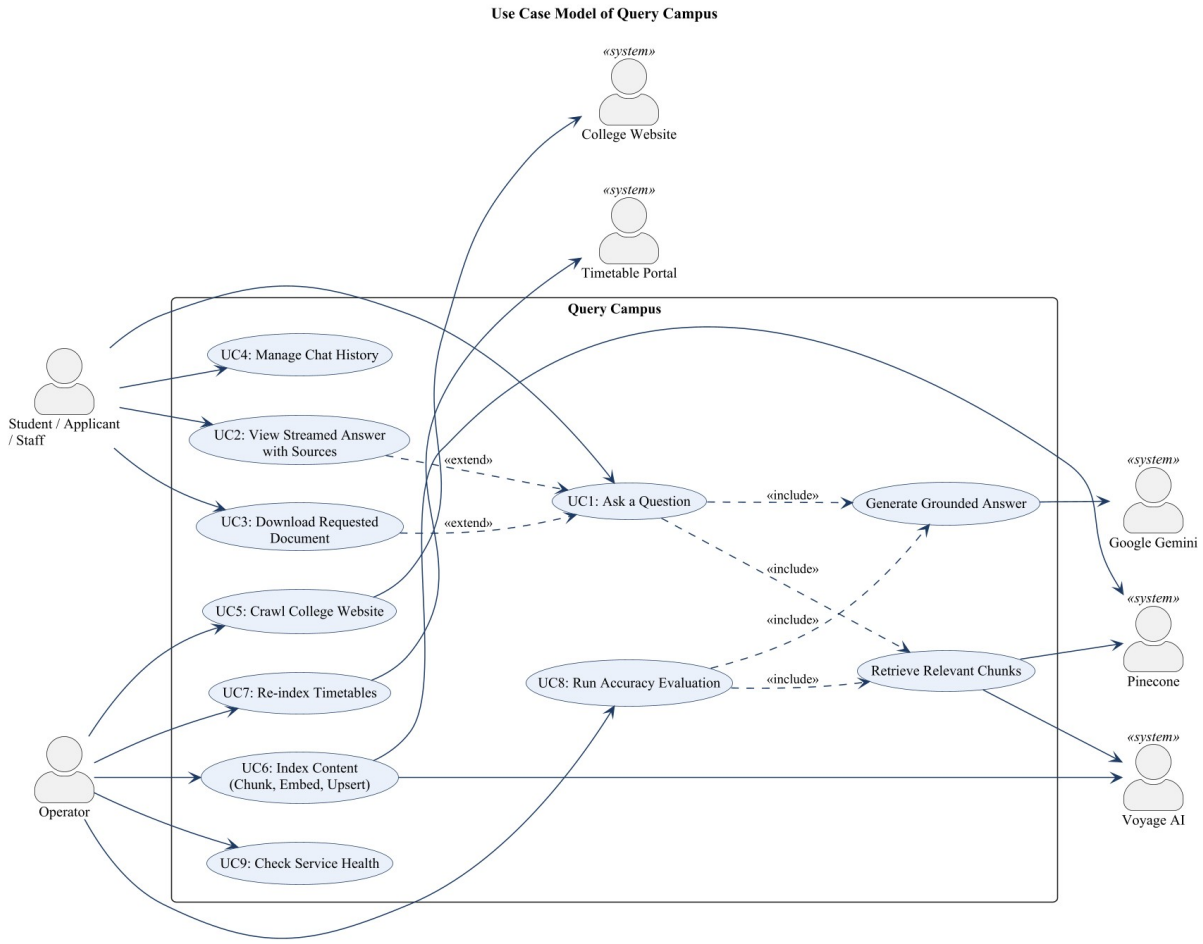


Fig. 3.1: Use case model of Query Campus

3.3.2 Use Case Description

Each use case is described below in the Rational format: name, short description, flow of events (basic and alternative), special requirements, preconditions, post conditions and extension points. The numbers are of the form 3.3.2.n.k, where n is the use case number and k is the item number.

3.3.2.1 UC1: Ask a Question

No.	Item	Description
3.3.2.1.1	Use Case Name	UC1: Ask a Question

No.	Item	Description
3.3.2.1.2	Brief Description	A student, applicant or staff member types a question in plain English; the system retrieves relevant content from the indexed college sources and produces a grounded, cited answer.
3.3.2.1.3.1	Basic Flow	1. The user types a question in the input box and presses Enter or the send button. 2. The chat interface creates a new chat if none is active, appends the question as a user message and shows a pending indicator. 3. The interface sends POST /query/stream with the question. 4. The backend validates and trims the question. 5. The system performs UC10 (Retrieve Relevant Chunks). 6. The system performs UC11 (Generate Grounded Answer). 7. The system streams the answer text to the interface, which reveals it progressively. 8. The system sends the final answer with its sources and attachments, and the interface saves the chat.
3.3.2.1.3.2	Alternative Flows	A1 Blank, missing or very long question: the interface ignores a blank submission; if the API is called directly it returns HTTP 400 (“question is required”, or “question is too long” for more than 500 characters). A2 No relevant content found: the system returns a fixed message asking the user to rephrase or visit the official site, without calling the language model. A3 Backend not configured: if an API key is missing the server returns a service-unavailable message, shown as an error message in the chat. A4 Provider failure: if all models fail after retries, the interface shows a generic error message and re-enables input. A5 User leaves the chat: the answer is stored in the chat that asked the question even if another chat is open. A6 User closes the page: the server detects the closed connection and stops generating. A7 Too many questions: a visitor who sends more than 20 questions in a minute gets HTTP 429 and a short message

No.	Item	Description
		to wait.
3.3.2.1.4	Special Requirements	Answers must start streaming without a login; if a model gives no first token within 12 seconds the next model is tried. Questions are expected in English.
3.3.2.1.5	Preconditions	The backend is running with valid API keys, the Pinecone index has been populated (UC5–UC7) and the user's browser can reach the backend.
3.3.2.1.6	Post Conditions	The assistant message is shown and saved in the browser's local storage. Nothing about the question is stored on the server.
3.3.2.1.7	Extension Points	View Answer with Sources (UC2); Download Requested Document (UC3).

3.3.2.2 UC2: View Streamed Answer with Sources

No.	Item	Description
3.3.2.2.1	Use Case Name	UC2: View Streamed Answer with Sources
3.3.2.2.2	Brief Description	While and after an answer is generated the user reads formatted text and can open the pages that support it.
3.3.2.2.3.1	Basic Flow	1. As delta events arrive, the interface removes citation markers and any partial ATTACH line from the text and shows it with a typewriter effect. 2. The text is rendered as markdown (paragraphs, bullet lists, bold text). 3. When the done event arrives, the interface replaces the text with the cleaned answer. 4. The interface lists each cited page under "Sources" as a link. 5. The user clicks a source link, which opens the original page in a new tab.
3.3.2.2.3.2	Alternative Flows	A1 No sources: when the context did not answer the question no source block is shown. A2 Error event: if the server sends an error event the interface shows the message in an error-styled bubble.

No.	Item	Description
3.3.2.2.4	Special Requirements	Duplicate sources are removed; sources that are shown as attachments are not repeated in the source list.
3.3.2.2.5	Preconditions	A question has been submitted (UC1).
3.3.2.2.6	Post Conditions	The complete answer and its links are visible and saved with the chat.
3.3.2.2.7	Extension Points	None.

3.3.2.3 UC3: Download Requested Document

No.	Item	Description
3.3.2.3.1	Use Case Name	UC3: Download Requested Document
3.3.2.3.2	Brief Description	When the user explicitly asks to get or send a document such as a timetable, syllabus or form, the answer includes a document card with Open and, for PDFs on the college site, Download actions.
3.3.2.3.3.1	Basic Flow	1. The user asks for a document, for example “Send me the 4th year CSE section B timetable”. 2. The system answers with a short summary and ends the reply with an ATTACH line naming the matching excerpts. 3. The backend maps those excerpts to attachments and sends them in the done event. 4. The interface shows a document card for each attachment. 5. The user clicks Open to view the document, or Download for a college PDF. 6. For Download the browser requests GET /documents/download?url=…, the backend validates the URL, streams the PDF from the college site and the browser saves it.
3.3.2.3.3.2	Alternative Flows	A1 URL is not a PDF on the college domain: the backend returns HTTP 400 “Only PDF documents from the college website can be downloaded”. A2 Malformed URL: the backend returns HTTP 400 “url

No.	Item	Description
		must be a valid URL”. A3 College site unreachable: the backend returns HTTP 502 with a readable message. A4 Attachment is a timetable portal page: only the Open action is shown.
3.3.2.3.4	Special Requirements	The download proxy must not work as an open proxy: only PDF files from the configured college host are allowed, with a 30-second timeout and a 50 MB size limit.
3.3.2.3.5	Preconditions	The index contains the requested document (UC5 or UC7).
3.3.2.3.6	Post Conditions	The user has opened or saved the document.
3.3.2.3.7	Extension Points	None.

3.3.2.4 UC4: Manage Chat History

No.	Item	Description
3.3.2.4.1	Use Case Name	UC4: Manage Chat History
3.3.2.4.2	Brief Description	The user starts new chats, switches between earlier chats and deletes chats. History is kept in the browser only.
3.3.2.4.3.1	Basic Flow	1. The user clicks “+ New chat” to start an empty conversation, or selects an earlier chat in the sidebar. 2. The interface loads the selected chat’s messages. 3. The user clicks the delete button of a chat. 4. The interface removes the chat and saves the remaining list.
3.3.2.4.3.2	Alternative Flows	A1 Storage unavailable: if local storage is blocked, chats work in memory but are lost on reload. A2 Narrow screens: the sidebar opens as a drawer.
3.3.2.4.4	Special Requirements	Chats are sorted by last activity; titles are the first question truncated to 60 characters.
3.3.2.4.5	Preconditions	The application is loaded in a browser.

No.	Item	Description
3.3.2.4.6	Post Conditions	The chat list in local storage reflects the user's actions.
3.3.2.4.7	Extension Points	None.

3.3.2.5 UC5: Crawl College Website

No.	Item	Description
3.3.2.5.1	Use Case Name	UC5: Crawl College Website
3.3.2.5.2	Brief Description	The operator runs the crawler, which collects text from HTML pages and PDF files of the college website into a JSON corpus.
3.3.2.5.3.1	Basic Flow	1. The operator runs "pnpm run scrape". 2. The system seeds the queue with the base URL and visits pages breadth-first, four at a time. 3. For an HTML page the system extracts same-site links, removes menus, scripts and footers, and records the title and the main-content text. 4. For a PDF link the system downloads the file and extracts its text. 5. The system stops when the queue is empty or 600 pages have been visited. 6. The system writes the documents to data/scraped.json and reports the count.
3.3.2.5.3.2	Alternative Flows	A1 A page fails or times out: a warning is logged and the crawl continues. A2 Non-HTML content type or text shorter than 50 characters: the page is skipped. A3 External domain link: ignored.
3.3.2.5.4	Special Requirements	The crawler identifies itself with a descriptive User-Agent, enforces 15 s / 20 s time-outs and 10 MB / 20 MB size limits.
3.3.2.5.5	Preconditions	The operator has installed the dependencies and configured SCRAPE_BASE_URL.

No.	Item	Description
3.3.2.5.6	Post Conditions	scraped.json contains the collected documents.
3.3.2.5.7	Extension Points	None.

3.3.2.6 UC6: Index Content (Chunk, Embed, Upsert)

No.	Item	Description
3.3.2.6.1	Use Case Name	UC6: Index Content (Chunk, Embed, Upsert)
3.3.2.6.2	Brief Description	The operator converts the scraped corpus into vectors and stores them in the vector database.
3.3.2.6.3.1	Basic Flow	1. The operator runs “pnpm run ingest” (or “pnpm run ingest --incremental” to add only the new pages). 2. The system reads scraped.json and any checkpoint file. 3. The system makes sure the Pinecone index exists, creating a serverless cosine index of 1024 dimensions if necessary. 4. The system splits each document into 1,500-character chunks with 200 characters of overlap. 5. For every batch of five chunks the system obtains embeddings from Voyage AI and upserts the vectors with their metadata into Pinecone. 6. After each batch the system writes a checkpoint and waits 21 seconds to respect the free-tier rate limit. 7. When the last batch is stored the checkpoint is deleted.
3.3.2.6.3.2	Alternative Flows	A1 HTTP 429, 5xx or network failure: the call is retried with increasing delay (up to 10 attempts), honouring Retry-After. A2 Run interrupted: the next run resumes from the checkpoint.
3.3.2.6.4	Special Requirements	Chunk identifiers are SHA-1 hashes of the URL and chunk number so re-indexing updates rather than duplicates.
3.3.2.6.5	Preconditions	scraped.json exists (UC5) and the Voyage and Pinecone keys are configured.

No.	Item	Description
3.3.2.6.6	Post Conditions	All chunks are searchable in the vector index.
3.3.2.6.7	Extension Points	None.

3.3.2.7 UC7: Re-index Timetables

No.	Item	Description
3.3.2.7.1	Use Case Name	UC7: Re-index Timetables
3.3.2.7.2	Brief Description	The operator refreshes the class and faculty timetables from the separate timetable portal each semester.
3.3.2.7.3.1	Basic Flow	1. The operator runs “pnpm run timetables”. 2. The system reads the available sessions and the programs of every semester from the portal and selects the latest session. 3. For each program the system fetches the timetables, parses the weekly grid and the subject codes. 4. The system creates one document per class and one per faculty member and saves them to timetables.json. 5. The system deletes the previous chunks whose id starts with “tt:” and indexes the new documents (UC6).
3.3.2.7.3.2	Alternative Flows	A1 “--dry-run”: the documents are scraped and saved but the index is not changed. A2 No session found: the run fails with an error message.
3.3.2.7.4	Special Requirements	Only timetable chunks are replaced; the rest of the index is unchanged. The keyword cache refreshes within ten minutes or on restart.
3.3.2.7.5	Preconditions	The timetable portal is reachable.
3.3.2.7.6	Post Conditions	The index contains the current semester’s timetables.
3.3.2.7.7	Extension Points	None.

3.3.2.8 UC8: Run Accuracy Evaluation

No.	Item	Description
3.3.2.8.1	Use Case Name	UC8: Run Accuracy Evaluation
3.3.2.8.2	Brief Description	The operator measures retrieval and answer accuracy on a fixed set of questions after changing the prompt, model or retrieval settings.
3.3.2.8.3.1	Basic Flow	1. The operator runs “pnpm run eval” (optionally with --retrieval-only or --only <text>). 2. For each case the system retrieves chunks (UC10) and checks that an expected page is among them and notes its rank. 3. Unless retrieval-only, the system generates the answer (UC11) and checks that required facts appear and that attachments are present only when expected. 4. The system prints a pass/fail line per case and a summary with latency. 5. The system compares the result with the previous run and lists regressions; the process exits with a non-zero code if any exist.
3.3.2.8.3.2	Alternative Flows	A1 Model quota exhausted: use retrieval-only mode, which does not call Gemini.
3.3.2.8.4	Special Requirements	Question embeddings are cached on disk so only the first run waits for the embedding rate limit.
3.3.2.8.5	Preconditions	The index is populated and eval/cases.json exists.
3.3.2.8.6	Post Conditions	A timestamped JSON report is stored and the latest baseline is updated for full runs.
3.3.2.8.7	Extension Points	None.

3.3.2.9 UC9: Check Service Health

No.	Item	Description
3.3.2.9.1	Use Case Name	UC9: Check Service Health

No.	Item	Description
3.3.2.9.2	Brief Description	The operator or a monitor checks that the backend is running.
3.3.2.9.3.1	Basic Flow	1. The operator sends GET /health. 2. The system returns HTTP 200 with {“status”: “ok”}.
3.3.2.9.3.2	Alternative Flows	A1 Server down: the request fails and no response is returned.
3.3.2.9.4	Special Requirements	No authentication and no external service calls.
3.3.2.9.5	Preconditions	None.
3.3.2.9.6	Post Conditions	None.
3.3.2.9.7	Extension Points	None.

3.3.2.10 UC10: Retrieve Relevant Chunks (included use case)

No.	Item	Description
3.3.2.10.1	Use Case Name	UC10: Retrieve Relevant Chunks (included use case)
3.3.2.10.2	Brief Description	Given a question, the system finds the most relevant indexed chunks using dense and keyword search.
3.3.2.10.3.1	Basic Flow	1. The system embeds the question with Voyage AI using the query input type (cached for repeated questions). 2. In parallel, the system queries Pinecone for the 25 nearest vectors and the BM25 index for the 25 best keyword matches. 3. The system fuses the two ranked lists with Reciprocal Rank Fusion (constant 60). 4. The system raises faculty pages when the question asks about a person, keeps at most three chunks from one page, and returns the best 16 chunks with their metadata.
3.3.2.10.3.2	Alternative Flows	A1 Keyword index not ready or not working: a warning is logged and only the search by meaning is used. A2 Search by meaning not available (for example the free limit of the embedding service is reached): only keyword

No.	Item	Description
		results are used. If both fail, the visitor sees a message that the search is busy. A3 No results: an empty list is returned.
3.3.2.10.4	Special Requirements	The keyword index is built in the background at start-up from a disk cache and refreshed every ten minutes without blocking requests.
3.3.2.10.5	Preconditions	The vector index and (optionally) the keyword cache exist.
3.3.2.10.6	Post Conditions	A ranked list of chunks is available to the caller.
3.3.2.10.7	Extension Points	None.

3.3.2.11 UC11: Generate Grounded Answer (included use case)

No.	Item	Description
3.3.2.11.1	Use Case Name	UC11: Generate Grounded Answer (included use case)
3.3.2.11.2	Brief Description	The system asks Gemini to write an answer using only the retrieved chunks.
3.3.2.11.3.1	Basic Flow	1. The system numbers the chunks and builds a context block with their URLs, titles and text. 2. The system sends the system instruction and the context plus question to the first available Gemini model. 3. The system waits up to 12 seconds for the first token and then streams the text. 4. When the stream ends, the system removes the ATTACH line, converts citation numbers to sources and attachments, and removes the markers from the text.
3.3.2.11.3.2	Alternative Flows	A1 Model rate-limited (429) or overloaded (5xx) or silent: the system tries the next model and avoids the failed model for 10 minutes (rate limit) or 1 minute (overload). A2 All models fail: the request is retried twice with back-off and then fails.
3.3.2.11.4	Special	The prompt forbids inventing URLs, dates or numbers,

No.	Item	Description
	Requirements	requires preferring the most recent session, and tells the model to attach documents only when explicitly asked.
3.3.2.11.5	Preconditions	Non-empty chunks were retrieved.
3.3.2.11.6	Post Conditions	The answer, sources and attachments are produced.
3.3.2.11.7	Extension Points	None.

3.4 Design Diagrams

3.4.1 Architecture Diagram

System overview

Several people and systems take part in Query Campus. **Students, applicants and staff** use the chat to ask questions. **The operator** runs the scrape, ingest and timetable commands when the college updates its content, and runs the test after changes. **The college web team** is an indirect stakeholder: the content they publish is the only source of truth, and our system never changes it. **Service providers** (Voyage AI, Pinecone, Google) give embedding, storage and answer-writing through APIs. **The guide and examiners** can check the quality of answers with the test report. The flow is simple: the operator fills the data from the college sources; the question goes from the browser to the backend; the backend uses Voyage AI and Pinecone to find evidence and Gemini to write the answer; the answer comes back to the browser with links to the college pages.

Architectural strategy

- **Layers and modules.** A screen layer (React), an application layer (NestJS controllers and the RAG service), an integration layer (clients for Voyage and Pinecone) and a separate indexing layer that runs from the command line using the same modules.
- **Offline and online parts.** Slow and limited work (crawling, embedding) is done offline. The online part only reads the index, so it is fast and the server stays simple.

- **Two kinds of search.** Search by meaning finds the topic, BM25 finds exact names and codes, and Reciprocal Rank Fusion merges both without any tuning.
- **Answers from evidence only.** The model gets only the found text and a prompt that asks for citations and refusal when evidence is missing. The server changes citation numbers into real links, so the model cannot invent links.
- **Keeps working.** Retry with waiting, backup models, a list of failing models, a first-word time limit and a 40-second overall limit keep answers coming on free plans. If the search by meaning is busy, the keyword search alone still answers, and the other way round.
- **Safety and privacy.** API keys stay on the server. The download route accepts only PDF files from the college site. Only the allowed website addresses can call the API, each visitor can ask 20 questions a minute, and a question can be at most 500 characters. No user data is stored on the server, and chat history stays in the user’s browser.

Block diagram and explanation

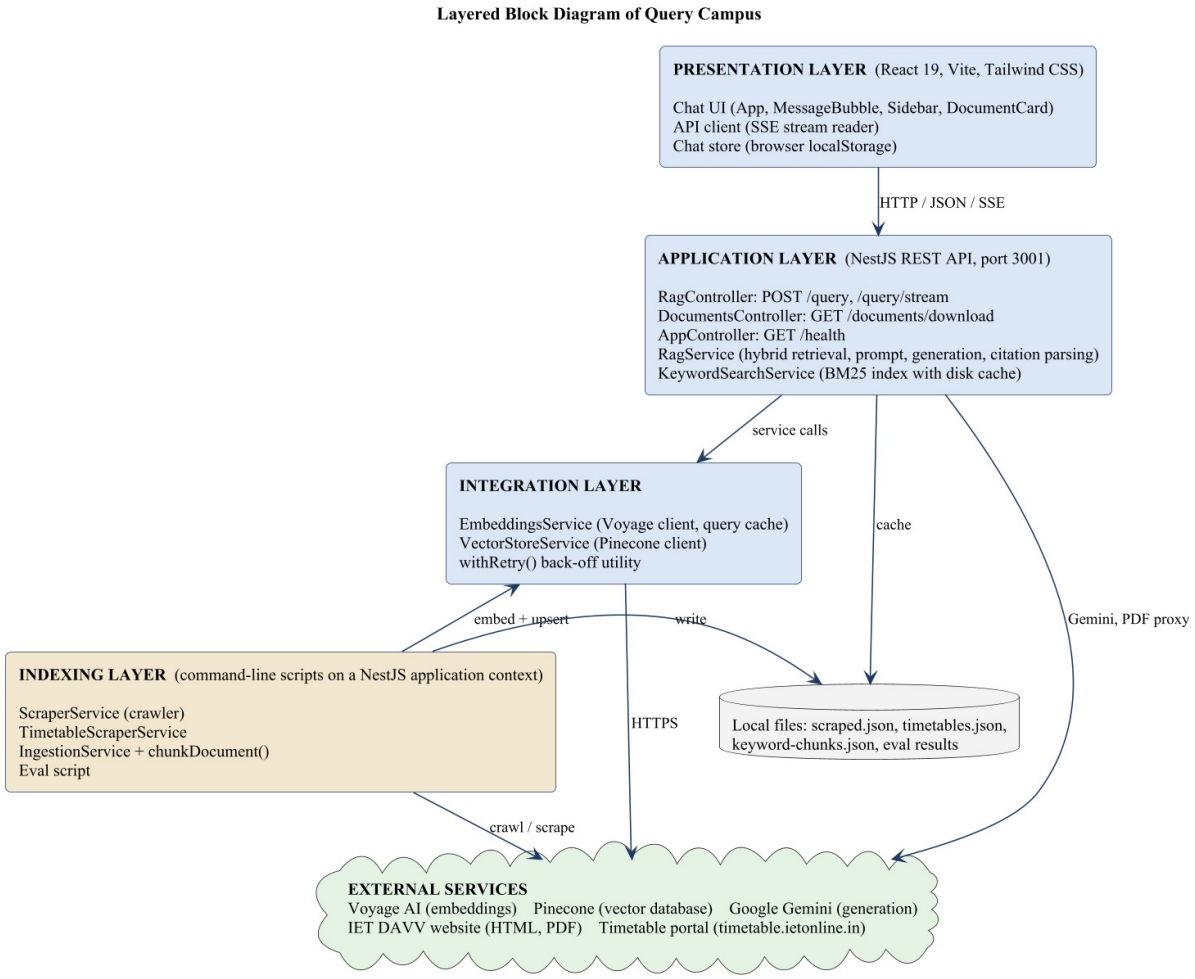


Fig. 3.2: Architecture (block) diagram of Query Campus

Screen layer. The chat screen (App, MessageBubble, Sidebar, DocumentCard, Composer) shows the conversation. The API client sends a POST request to /query/stream and reads the events. The chat store keeps chats in the browser's local storage.

Application layer. RagController checks the request and sends the events. DocumentsController allows safe PDF downloads. AppController reports health. RagService does the search, builds the prompt, streams from Gemini and cleans the text into the final answer. KeywordSearchService keeps the BM25 index of all chunks, starting from a copy on disk. At start-up, main.ts also sets the allowed website addresses (CORS_ORIGIN) and the rate limit.

Integration layer. EmbeddingsService talks to Voyage AI and remembers question vectors. VectorStoreService talks to Pinecone (create index, save, search, list, fetch, delete by prefix). withRetry gives one common retry rule.

Indexing layer. ScraperService crawls the college site. TimetableScraperService reads the timetable portal. IngestionService cuts, embeds and saves. The test script runs RagService on saved questions. These run from the command line and cannot be reached over HTTP.

Outside services and local files. Voyage AI, Pinecone and Gemini are called over HTTPS. The college sites are only read. Local JSON files keep the scraped data, the timetable documents and the keyword cache.

3.4.2 Sequence Diagrams

Fig. 3.3 shows how a question is answered. After the controller checks the question, RagService runs the two searches at the same time and merges the results. Then it either returns the fixed “nothing relevant found” message or streams the answer from Gemini. Each piece of text goes to the browser as a delta event. At the end, the final answer, sources and attachments are sent as a done event.

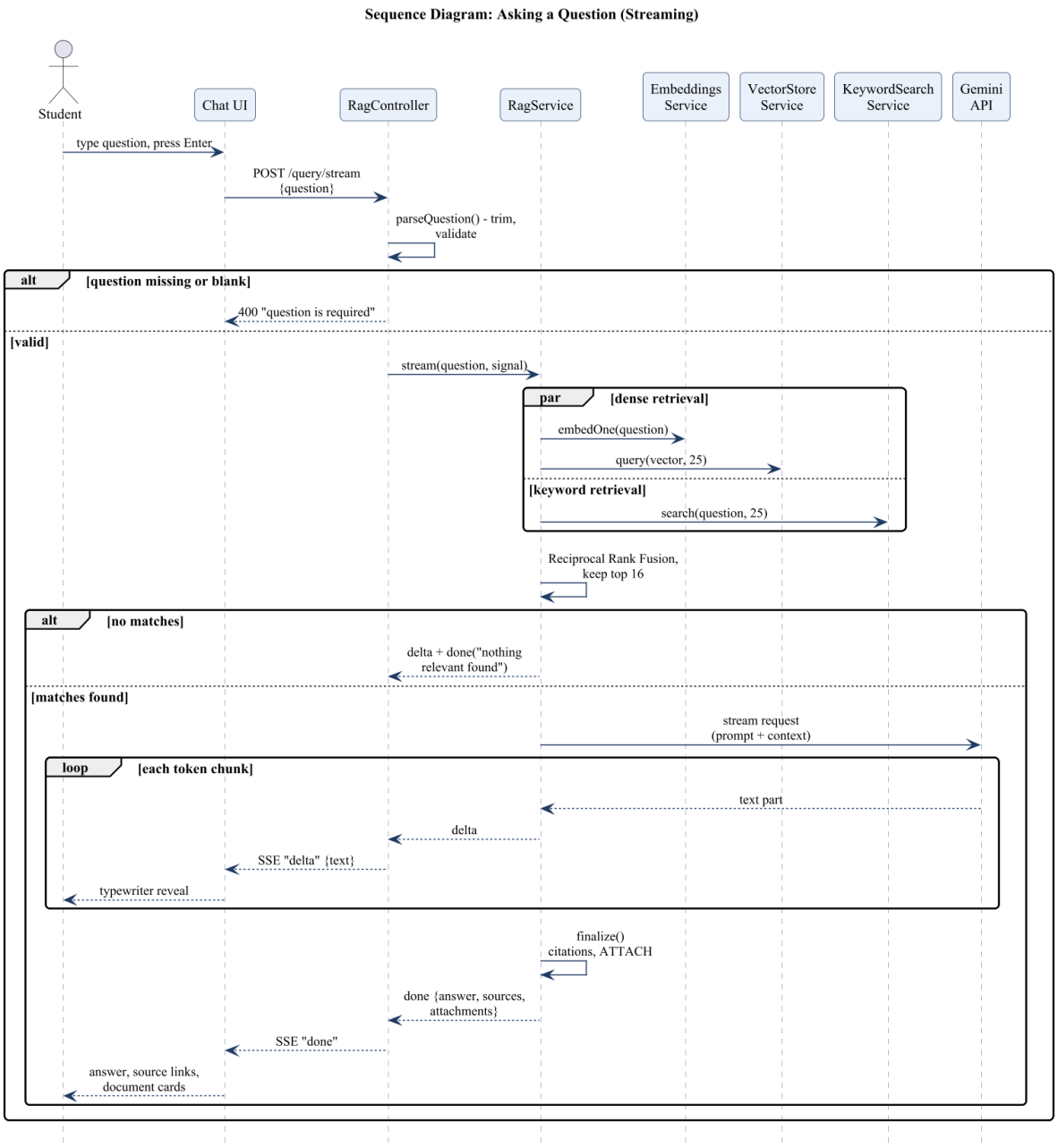


Fig. 3.3: Sequence diagram for asking a question

Fig. 3.4 shows indexing. The operator first crawls the website into scraped.json. The ingest command then reads the data and any checkpoint, makes sure the index exists, cuts the documents, and for every batch of five chunks gets embeddings, saves them, writes a checkpoint and waits to respect the rate limit.

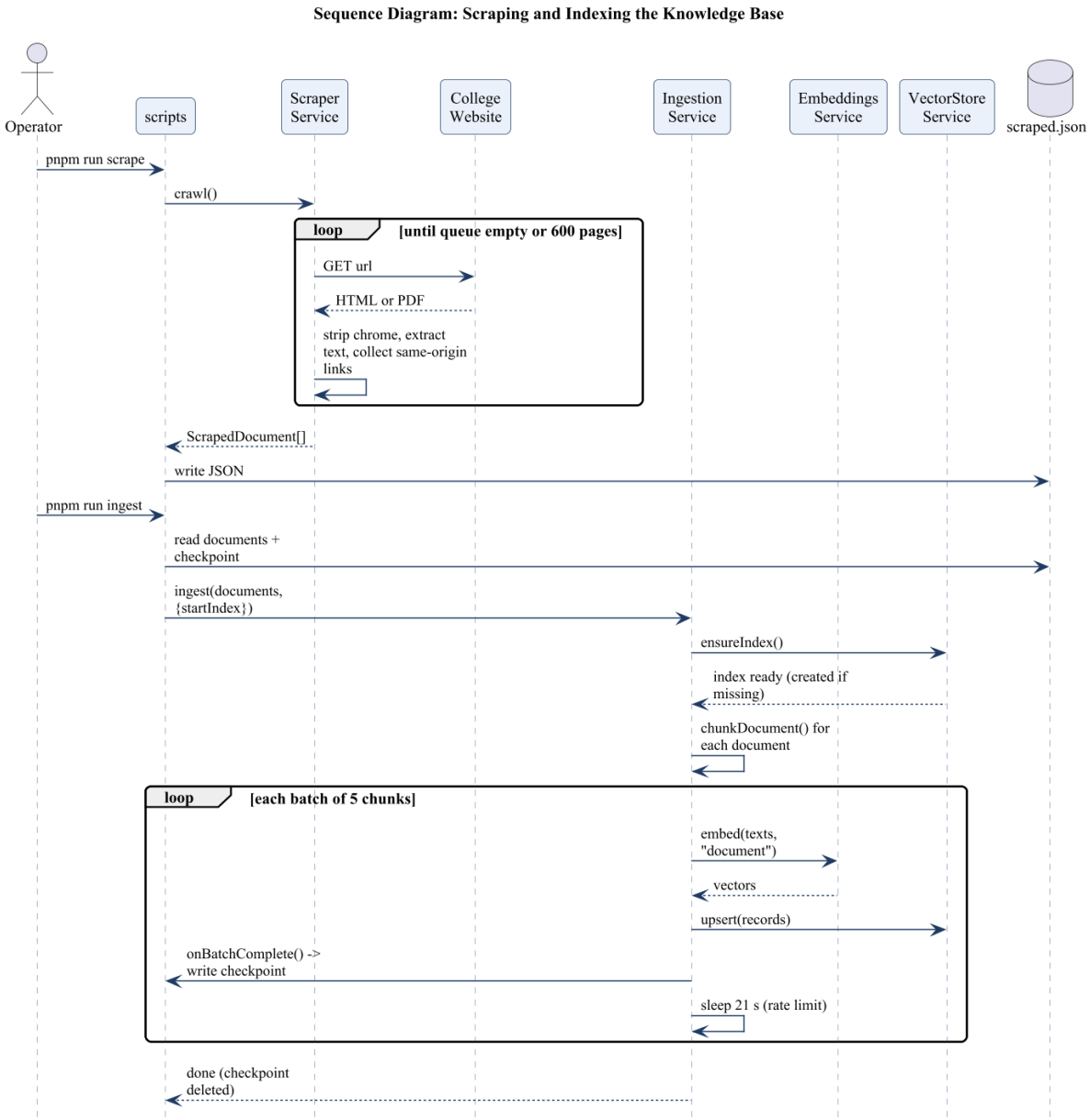


Fig. 3.4: Sequence diagram for scraping and indexing

Chapter-4 Implementation and Testing

4.1 Database Design

Query Campus does not use a relational (SQL) database. Its main data is a **vector index in Pinecone**, backed by JSON files on the backend computer, and chat history is kept in the **browser (localStorage)**. Fig. 4.1 shows these entities with their cardinalities. The tables after it show the fields of each entity, taken from the source code, with a short description. One scraped document is cut into one or more chunks (1 to many). Each chunk has at most one keyword-cache entry. A chat has zero or more messages. A message has zero or more sources, and each source points to exactly one document. A test case expects zero or more documents to be found.

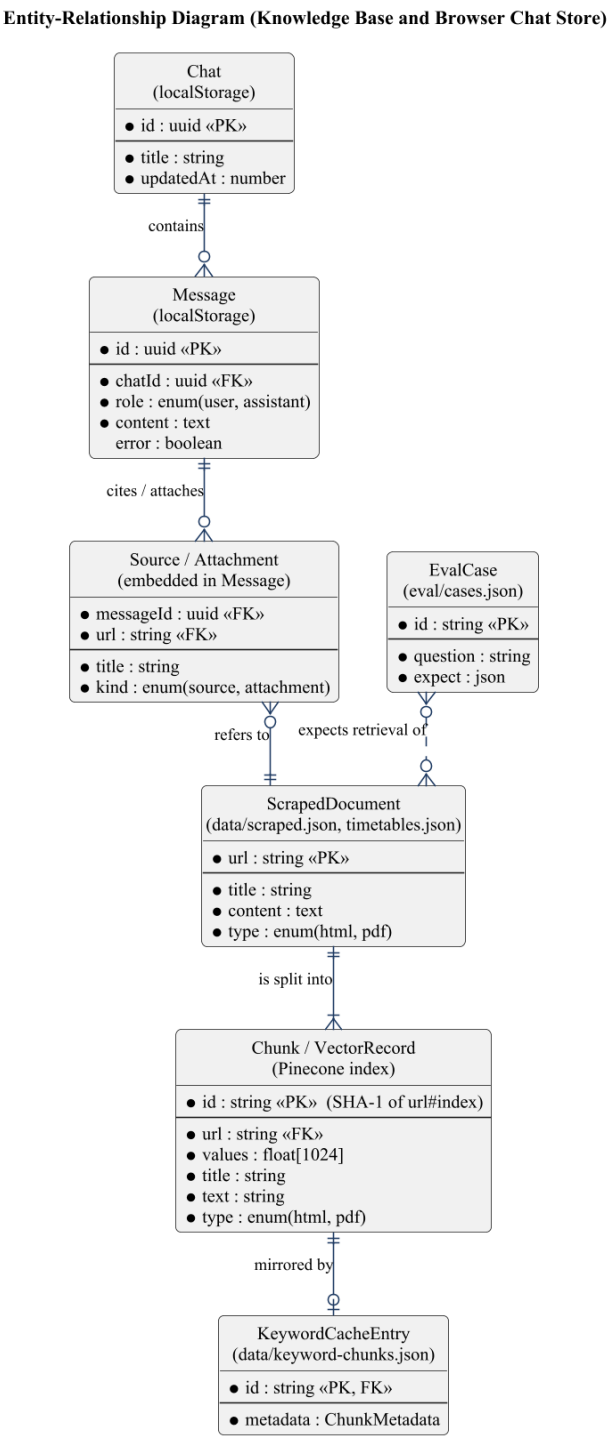


Fig. 4.1: Entity-relationship diagram of Query Campus

Table 4.1: ScrapedDocument (data/scraped.json and data/timetables.json)

Field	Type	Key	Description
url	string	PK	Absolute, normalised address of the page or PDF (timetable documents use a fragment such as #faculty-name).

Field	Type	Key	Description
title	string		Page title, PDF file name, or generated title for timetable documents.
content	text		Cleaned text with whitespace collapsed to single spaces.
type	enum		“html” or “pdf”.

One entry for every crawled HTML page or PDF and every generated timetable document. It is the raw material for chunking and is rewritten each time the crawler or timetable scraper runs.

Table 4.2: Chunk / VectorRecord (Pinecone index “query-campus”)

Field	Type	Key	Description
id	string	PK	SHA-1 hash of “url#chunkNumber”, optionally prefixed (“tt:” for timetables), so re-indexing overwrites instead of duplicating.
values	float[1024]		Embedding vector produced by Voyage AI (voyage-3); cosine metric.
url	string	FK	Address of the source document (metadata).
title	string		Title of the source document (metadata).
text	string		The chunk text shown to the language model (metadata).
type	enum		“html” or “pdf” (metadata).

One record per chunk of at most 1,500 characters. The vector is used for semantic search and the metadata is returned with each match, so no separate lookup is needed to build citations. The index now holds 2,864 records, of which 350 are timetable chunks (ids prefixed “tt:”).

Table 4.3: KeywordCacheEntry (data/keyword-chunks.json)

Field	Type	Key	Description
id	string	PK, FK	Chunk id, same as the Pinecone record id.
metadata	object		url, title, text and type of the chunk.

A disk copy of every chunk's id and metadata, used to build the in-memory BM25 index quickly. It is synchronised with Pinecone: new ids are downloaded, deleted ids are dropped.

Table 4.4: Chat (browser localStorage, key "query-campus.chats")

Field	Type	Key	Description
id	uuid	PK	Generated with crypto.randomUUID().
title	string		First question, truncated to 60 characters.
updatedAt	number		Time of the last change (milliseconds).
messages	Message[]		Messages in order of creation.

A conversation. The list of chats is stored as one JSON array and is sorted by most recent activity.

Table 4.5: Message, Source and Attachment (embedded in a Chat)

Field	Type	Key	Description
id	uuid	PK	Message identifier.
role	enum		"user" or "assistant".
content	text		Question text, or the cleaned markdown answer.
sources	RagSource[]	FK	Cited pages: url, title, type.
attachments	RagSource[]	FK	Documents the user asked for: url, title, type.

Field	Type	Key	Description
error	boolean		True when the message is an error notice.

One entry per user question or assistant reply. Sources and attachments are stored inside the assistant message that produced them and refer to documents by URL.

Table 4.6: EvalCase (apps/backend/eval/cases.json)

Field	Type	Key	Description
id	string	PK	Short name, for example “class-coordinator-iv-cse-b”.
question	string		The question asked.
expect.retrieval	string[]		URL fragments, one of which must appear in the retrieved chunks.
expect.answer	object		Words that must appear (arrays mean any one of them) or must not appear.
expect.attachments	string		“none”, or groups of title/URL fragments that must be attached.

A test question with the facts and pages that a correct run must contain. The file holds test cases for faculty, timetables, fees, branches, document requests and an unanswerable question.

4.2 Class Diagram

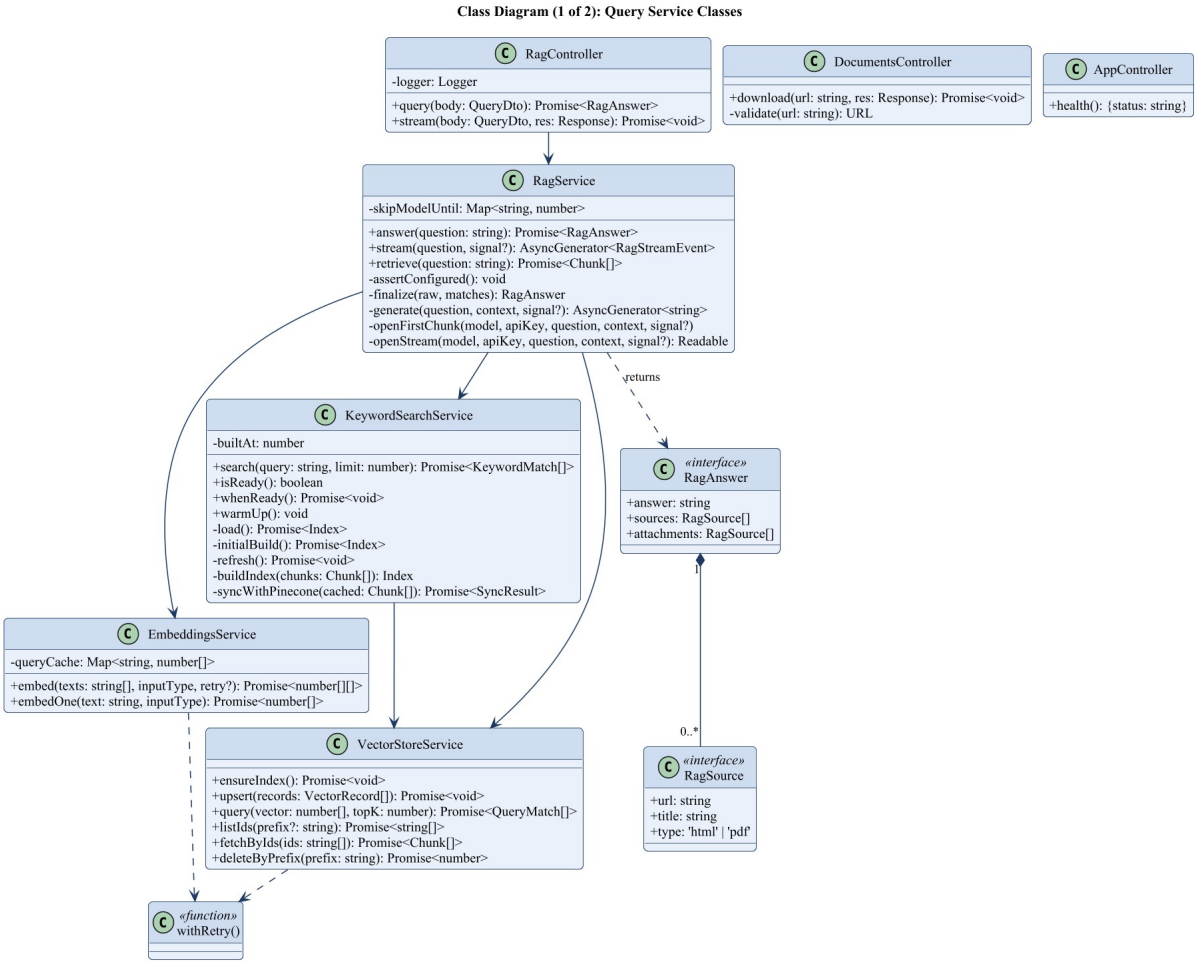


Fig. 4.2: Class diagram (1 of 2): query service classes

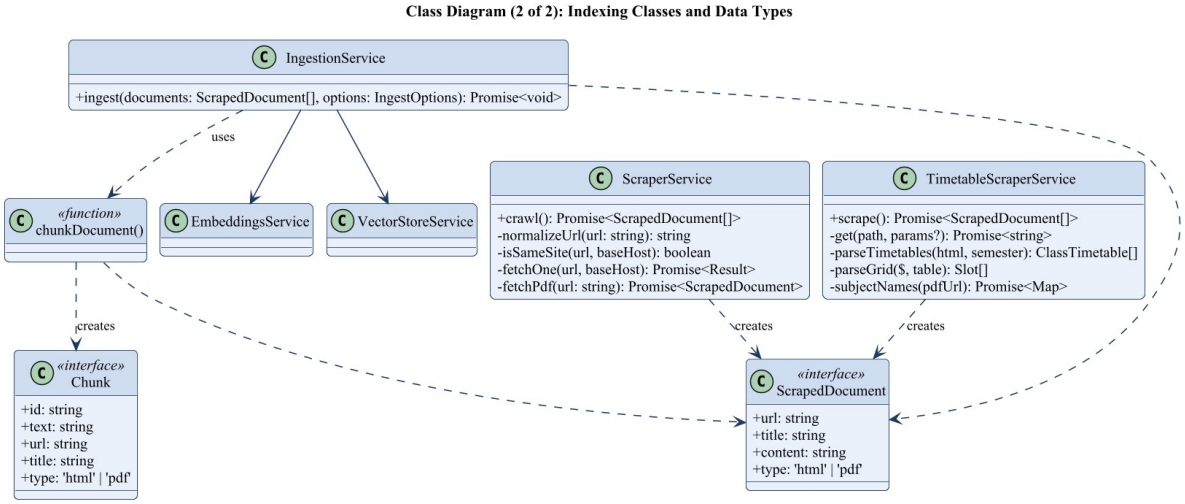


Fig. 4.3: Class diagram (2 of 2): indexing classes and data types

Explanation of the classes

The following explains the purpose of each class of Figs. 4.2 and 4.3 and the working of its methods together with the parameters they take. The classes are NestJS providers or controllers unless stated otherwise; the interfaces at the end describe the shared data structures.

4.2.1 RagController

Purpose. HTTP entry point of the question-answering service. It validates input and converts the answer into either a JSON response or a stream of server-sent events.

Method	Parameters	Working
query	body: QueryDto { question }	Checks the question with parseQuestion (HTTP 400 “question is required” if it is missing, not text or blank, and “question is too long” above 500 characters) and returns RagService.answer(question) as one JSON object.
stream	body: QueryDto, res: Response	Validates the question, sets text/event-stream headers, creates an AbortController that fires when the client disconnects, then iterates RagService.stream(). Each text piece is sent as a “delta” event and the final result as a “done” event; failures are sent as an “error” event with a user-safe message.

4.2.2 DocumentsController

Purpose. Serves college PDF files as downloads, because browsers ignore the download attribute on cross-origin links.

Method	Parameters	Working
download	url: string (query), res: Response	Validates the URL, streams the PDF from the college site (30 s time-out, 50 MB limit) to the client with a Content-Disposition attachment header and sanitised file name; responds with HTTP 502 and a message if the college site cannot be reached.
validate	url: string | undefined	Parses the URL and accepts it only if the protocol is http/https, the host (without "www.") equals the configured college host and the path ends in .pdf; otherwise throws BadRequestException. This prevents use as an open proxy.

4.2.3 AppController

Purpose. Provides the health-check endpoint.

Method	Parameters	Working
health	none	Returns { status: "ok" } for GET /health.

4.2.4 RagService

Purpose. The core of the system: it retrieves evidence, asks Gemini for an answer constrained to that evidence, and post-processes the text into answer, sources and attachments.

Method	Parameters	Working
answer	question: string	Consumes stream() until the "done" event and returns it as a single RagAnswer.
stream	question: string, signal?: AbortSignal	Async generator. Checks configuration, calls retrieve(); if there are no matches yields a fixed message; otherwise builds the numbered context block, yields each text piece from generate() as a delta event and finally yields a done event produced by finalize(). Logs retrieval and first-token timings.

Method	Parameters	Working
retrieve	question: string	Runs search by meaning (embedOne + Pinecone, 25 results) and BM25 keyword search (25 results) at the same time. If one of them fails or is not ready, the other is used; if both fail, a “search is busy” message is returned. For questions about a person, faculty pages get a boost. The two lists are merged with Reciprocal Rank Fusion (score += 1/(60 + rank + 1)), at most 3 chunks per page are kept, and the best 16 are returned. It is public so the test tool can check search alone.
assertConfigured	none	Throws ServiceUnavailableException naming any missing key among GEMINI_API_KEY, VOYAGE_API_KEY and PINECONE_API_KEY.
finalize	raw: string, matches: chunk list	Splits off the trailing “ATTACH: [n]” line, maps bracketed numbers to unique sources and attachments by URL, removes the markers and stray spaces from the text, and removes attached documents from the source list.
generate	question, context: string, signal?: AbortSignal	Orders the configured models so recently failed ones go last, opens a stream with retry (2 retries, 3 s base delay), records failures in a skip map (10 minutes for 429, 1 minute for overload or silence) and parses the server-sent events, yielding only non-thought text parts.
openFirstChunk	model, apiKey, question, context: string, signal?	Opens the stream for one model and waits at most 12 seconds for the first chunk; on time-out aborts the connection and throws FirstTokenTimeout so the next model is tried.

Method	Parameters	Working
openStream	model, apiKey, question, context: string, signal?	Posts to the Gemini streamGenerateContent endpoint with the system prompt, the context and question, maxOutputTokens 2048 and the configured thinking level; returns the response stream.

4.2.5 KeywordSearchService

Purpose. Maintains an in-memory BM25 index (MiniSearch) over all chunks to find exact names, codes and numbers.

Method	Parameters	Working
search	query: string, limit: number	Adds the college wording to the question (“2nd year IT” also searches “II Year Information Technology”), makes spelling variants equal, runs an AND query (chunks with every word) and then an OR query, and returns the best results with their metadata. Stop-words are ignored, fuzzy and prefix matching apply to words of five or more letters, and title and page-address words count more.
warmUp	none	Builds the index in the background at server start so the first question does not wait.
isReady	none	True when an index exists. The question search uses it to skip the keyword search instead of waiting for a first build.
whenReady	none	Waits until the index can be searched. The test tool uses it so that all cases run with keyword search.
load	none	Returns the cached index promise and, if it is older than ten minutes, rebuilds it in the background (stale-while-revalidate).

Method	Parameters	Working
initialBuild	none	Builds the index from the copy on disk at once, so the server answers well right after a restart, and then syncs with Pinecone in the background. Without a copy it downloads all chunks first.
syncWithPinecone	cached chunks	Lists all ids in Pinecone, downloads only the chunks that are missing, drops deleted ones and rewrites the copy on disk. The index is rebuilt only if something changed.

4.2.6 EmbeddingsService

Purpose. Client for the Voyage AI embedding API with retry and a small cache for question vectors.

Method	Parameters	Working
embed	texts: string[], inputType: “query” | “document”, retry?	Sends the texts to the Voyage embeddings route (30 s time-out), puts the answer back in order and returns the vectors. It retries on HTTP 429, 5xx and network errors, honouring Retry-After. The number of retries can be set by the caller.
embedOne	text: string, inputType	Returns one vector. For questions it first checks an in-memory cache of up to 500 questions. A question vector is tried only once: if the free limit is reached, the caller switches to keyword search at once instead of waiting.

4.2.7 VectorStoreService

Purpose. Client for the Pinecone index.

Method	Parameters	Working
ensureIndex	none	Creates the serverless cosine index with the configured dimension if it does not exist and waits until ready.

Method	Parameters	Working
upsert	records: VectorRecord[]	Writes records in batches of 100 with retry.
query	vector: number[], topK = 6	Returns the topK nearest records with metadata and scores.
listIds	prefix?: string	Pages through the ids of all records, optionally with a prefix.
fetchByIds	ids: string[]	Fetches the metadata of the given ids in pages of 100.
deleteByPrefix	prefix: string	Lists all ids with the prefix, then deletes them in groups of 1,000; returns how many were removed. Used to replace the timetable chunks.

4.2.8 IngestionService

Purpose. Turns scraped documents into indexed vectors.

Method	Parameters	Working
ingest	documents: ScrapedDocument[], options: { startIndex?, idPrefix?, skipExisting?, onBatchComplete? }	Makes sure the index exists, cuts all documents into chunks, and with skipExisting leaves out chunks that are already in the index. Then, for each batch (5 chunks by default), it embeds the chunks as “document”, saves the vectors with their data, calls onBatchComplete (used for the checkpoint) and waits (21 seconds by default). Batch size and wait are set by EMBED_BATCH_SIZE and EMBED_BATCH_DELAY_MS, so a paid plan can index much faster.

4.2.9 ScraperService

Purpose. Crawls the college website.

Method	Parameters	Working
crawl	none	Breadth-first traversal from the base URL with a visited set and a queue, four requests at a time, until the queue is empty or the page limit is reached; returns the collected ScrapedDocuments.
normalizeUrl	url: string	Removes the fragment and any trailing slash of non-root paths.
isSameSite	url, baseHost: string	True if the host without “www.” equals the base host.
fetchOne	url, baseHost: string	For a PDF delegates to fetchPdf; for HTML downloads the page, collects same-site links (skipping mailto, tel, javascript and static file types), removes page chrome, extracts the main content (#sp-component or body) and returns the document (if longer than 50 characters) and links. Failures are logged, not thrown.
fetchPdf	url: string	Downloads the file, extracts its text with pdf-parse, collapses whitespace and returns a pdf document, or null if there is almost no text.

4.2.10 TimetableScraperService

Purpose. Converts the timetable portal into searchable text.

Method	Parameters	Working
scrape	none	Finds the latest session and the programs of each semester, fetches and parses every program’s timetables, reads subject names from the printable view and returns one document per class plus one per faculty member.
get	path: string, params?: object	HTTP GET to the portal with a User-Agent and a long time-out; returns HTML.

Method	Parameters	Working
parseTimetables	html: string, semester: string	Parses the page (after removing embedded images) into ClassTimetable records with program, branch, year, section, room, coordinator and slots.
parseGrid	$: CheerioAPI, table	Reads a weekly grid into slots of day, time, subject, kind, faculty, room and batch.
subjectNames	pdfUrl: string	Reads the printable view to map subject codes to subject names.

4.2.11 Helper functions

Purpose. These plain functions are used by several classes.

Method	Parameters	Working
chunkDocument	doc: ScrapedDocument, idPrefix = “”	Slides a window of 1,500 characters over the content with 200 characters of overlap, trims it, skips empty slices and assigns each chunk the SHA-1 id of “url#index”.
withRetry	fn, options { label, logger, maxRetries = 10, baseDelayMs = 21,000, isRetryable, retryAfterMs }	Calls fn; on a retryable error waits for Retry-After or an increasing delay (capped at 90 s), logs a warning and retries; rethrows when the error is not retryable or attempts are exhausted.
normalizeText	text: string	Makes spelling variants equal: “time table”, “time-table” and “timetable” become one word, and the same for “date sheet”.
expandQuery	query: string	Adds the college wording to a class question: “2nd year” becomes “II Year”, and “IT”, “CSE” and similar short names get their full names. The original question is kept.
urlWords	url: string	Turns the path of a page address into words, so that a page called exam-time-table can be found by those words.

Method	Parameters	Working
limitPerKey	items, key, max	Keeps the order of the items but allows at most max items per key. RagService uses it to keep at most 3 chunks from one page.
rateLimit	maxPerMinute: number	Returns a middleware that counts questions per visitor address. After the limit it answers HTTP 429 with a Retry-After header. A value of 0 switches it off.

Data interfaces. ScrapedDocument (url, title, content, type) is produced by the scrapers; Chunk (id, text, url, title, type) is produced by chunkDocument; VectorRecord (id, values, metadata) is stored in Pinecone; RagSource (url, title, type) describes a page or document; RagAnswer (answer, sources, attachments) is the final result of a question.

4.3 Test Cases

4.3.1 Testing techniques used

- **Black-box testing** of the REST routes and the screen: we give inputs and compare the outputs with what is expected, without looking at the code.
- **Invalid input testing:** blank, empty, missing and very long questions, and wrong, non-PDF or foreign download links.
- **Unit tests** (13 tests) for the chunker, the text helpers (spelling variants, year and branch words, page limit) and the rate limiter.
- **End-to-end tests** (10 tests) with the NestJS testing module and Supertest. They start the whole application and check the health route and all input checks.
- **Accuracy testing** with 18 real questions. For each question the test checks that a page holding the answer was found (and at which rank), that the required facts are in the answer, and that a document is attached only when the user asked for it. Each run is compared with the previous run to find regressions.
- **Stress and failure testing:** six different questions were sent one after another to hit the free limit of the embedding service; the system switched to keyword search and still

answered in about two seconds. The backend was also stopped while the page was open, to check the error message.

- **Manual end-to-end testing** in the browser on a computer-size and a phone-size screen: suggestion cards, typed questions, follow-up buttons, document cards, saved chats, deleting a chat and reloading the page.

The route and screen tests below were run on 22 and 23 October 2026 on the running application (backend on port 3001, frontend on port 5173) with the final knowledge base of 2,864 chunks. The accuracy test was run on 23 October 2026 with the model gemini-3.5-flash-lite.

4.3.2 Screen snapshots and test cases

Snapshot 1: Home screen. The home screen has a cream page with a lighter side panel for saved chats. In the middle are a small status label, the heading “Ask about” with a word that changes every two seconds, one line about the sources, four suggestion cards, a slowly moving row of topic buttons and a round question box.

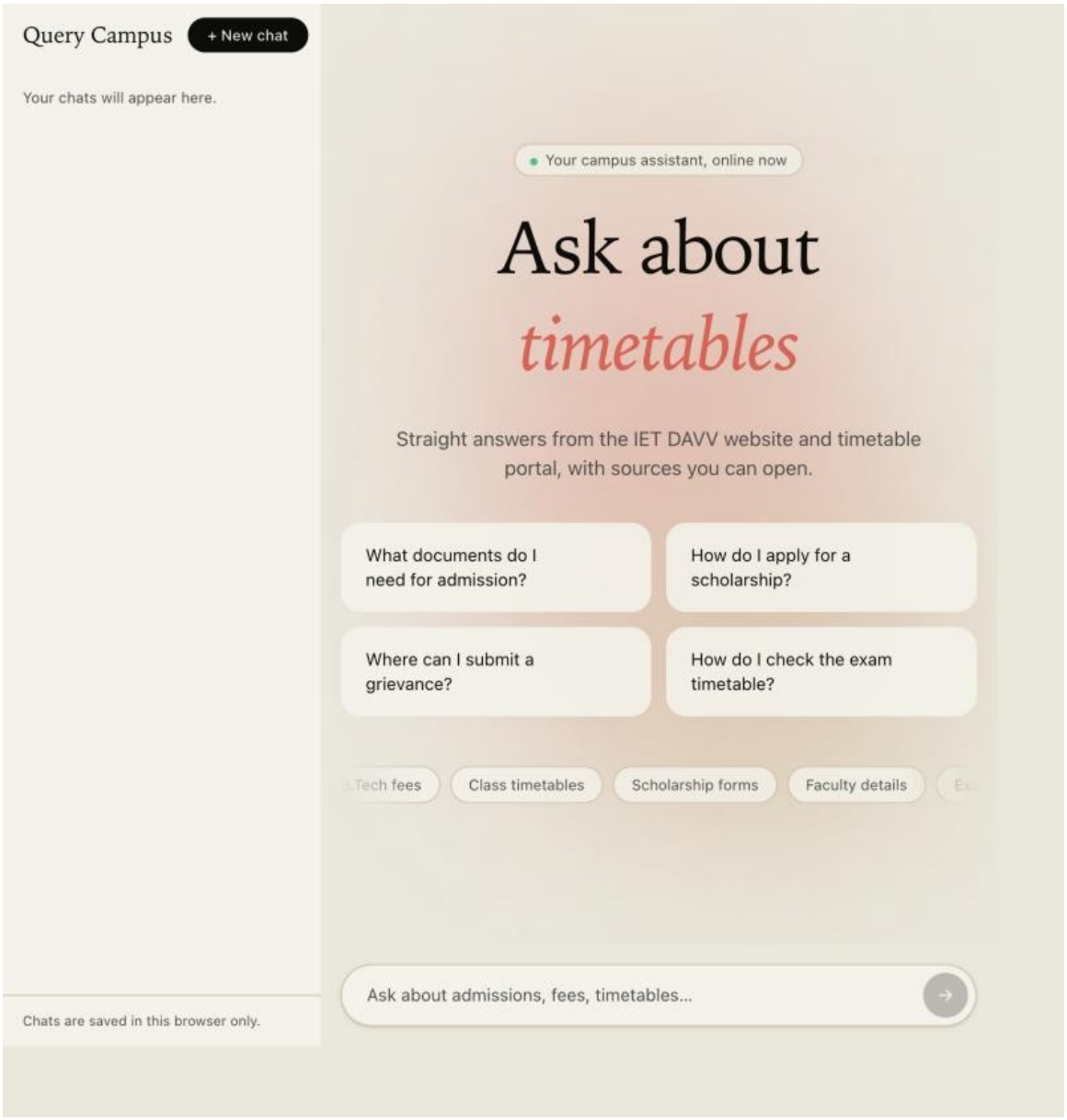


Fig. 4.4: Home screen of Query Campus

Test 01: Test case for loading the application

Input: Open http://localhost:5173 in a browser

System output: Home screen with four suggestion cards and the question box

Result: Passed

Test 02: Test case for the health route

Input: GET /health

System output: HTTP 200, body {“status”:“ok”}

Result: Passed

Snapshot 2: A question answered from the timetable portal. The user asks for a class coordinator. The answer is short and correct, the timetable page that was used is shown under “Sources”, and three ready questions appear below.

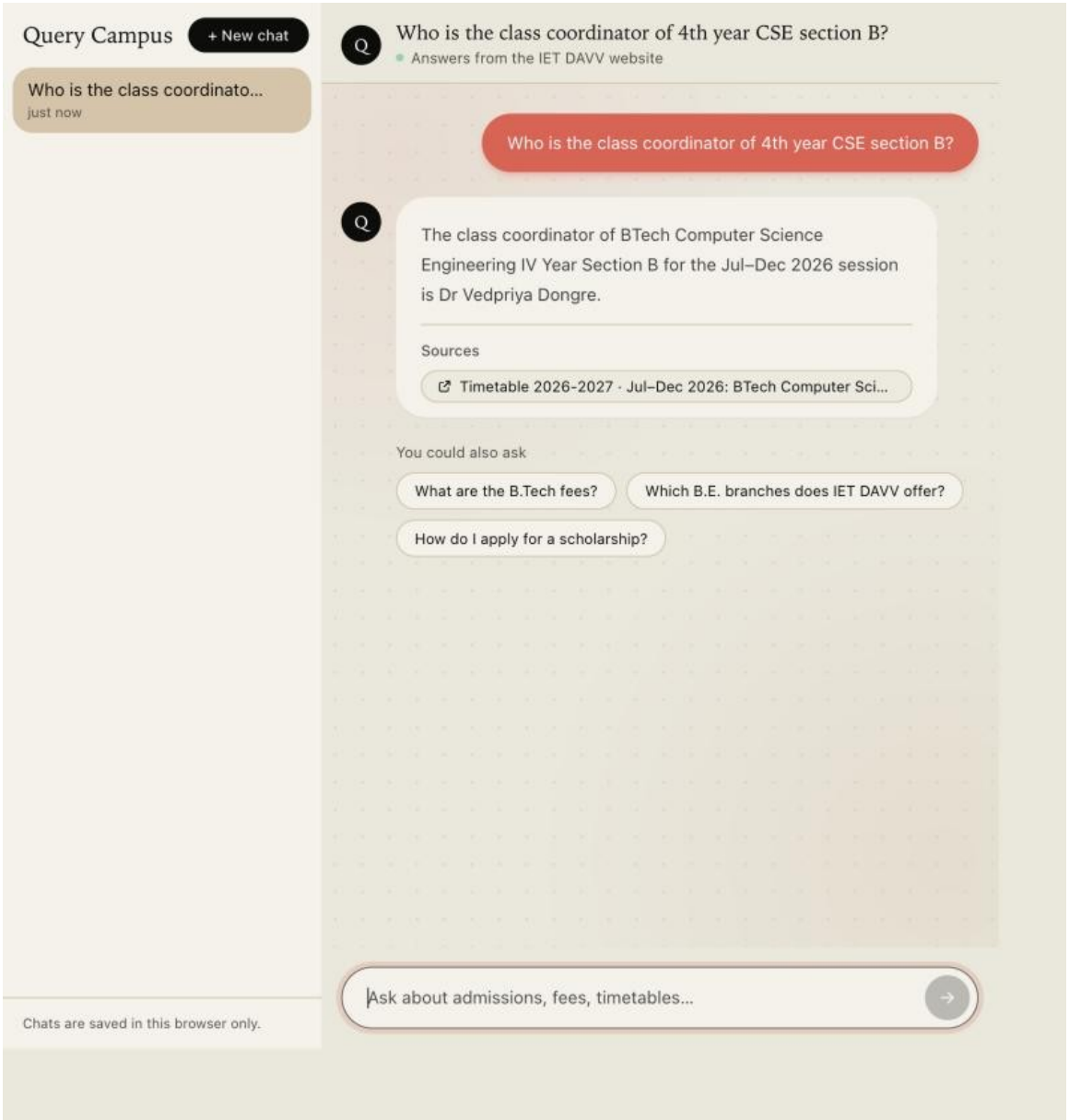


Fig. 4.5: Answer with a source link

Test 03: Test case for a valid question

Input: Who is the class coordinator of 4th year CSE section B?

System output: The class coordinator of BTech Computer Science Engineering IV Year Section B for the Jul–Dec 2026 session is Dr Vedpriya Dongre; source: timetable of BTech Computer Science Engineering IV Year Section B

Result: Passed

Snapshot 3: Document request. When the user asks the system to send a document, the answer gives the main details and shows a document card with an Open button.

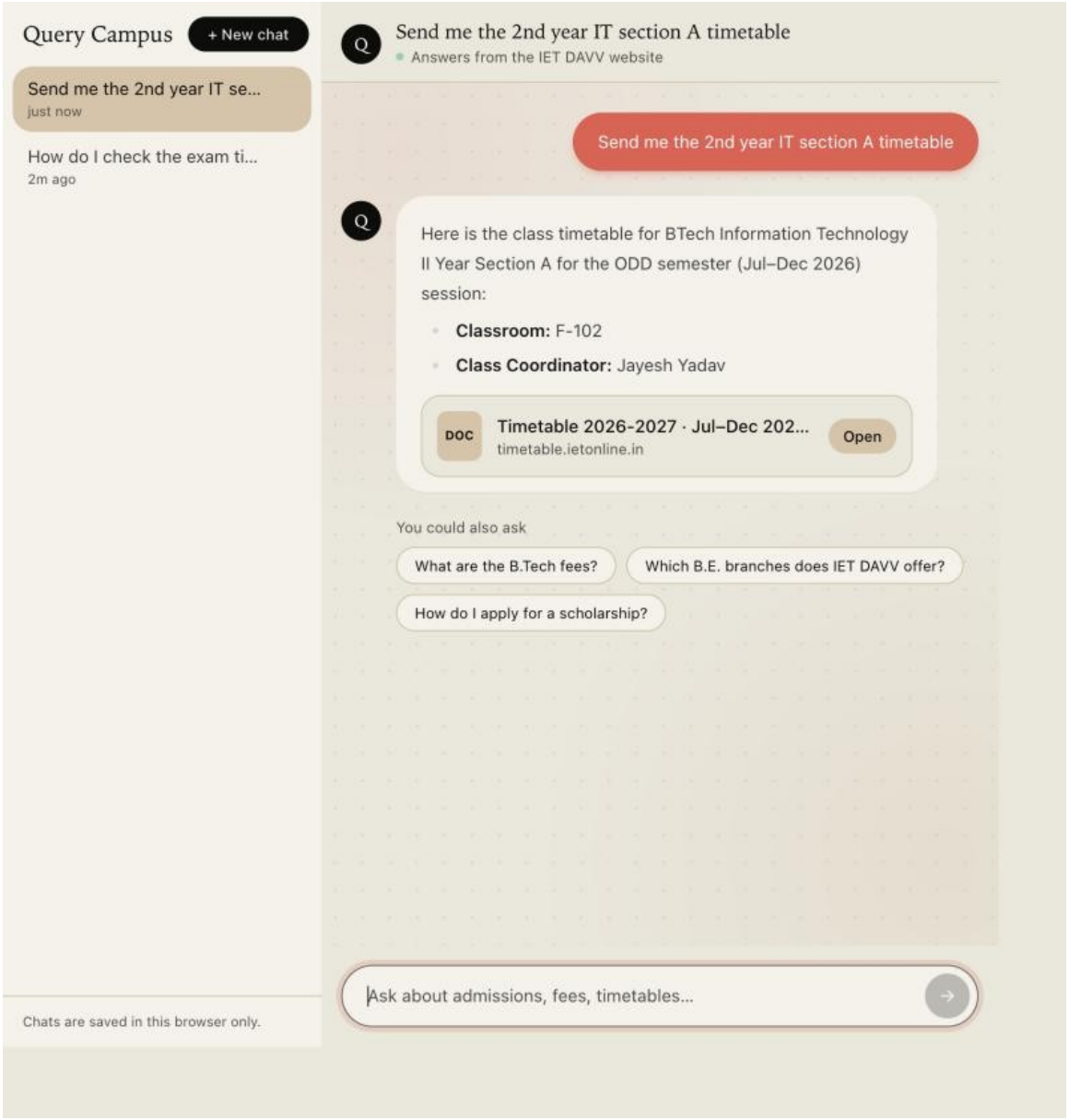


Fig. 4.6: Answer with a document card

Test 04: Test case for a document request

Input: Send me the 2nd year IT section A timetable

System output: Details (BTech Information Technology II Year Section A, classroom F-102, class coordinator Jayesh Yadav) and a document card for the timetable on timetable.ietonline.in with an Open button

Result: Passed

Snapshot 4: A question the sources cannot answer. The assistant says it could not find it, instead of making up an answer. No sources are shown.

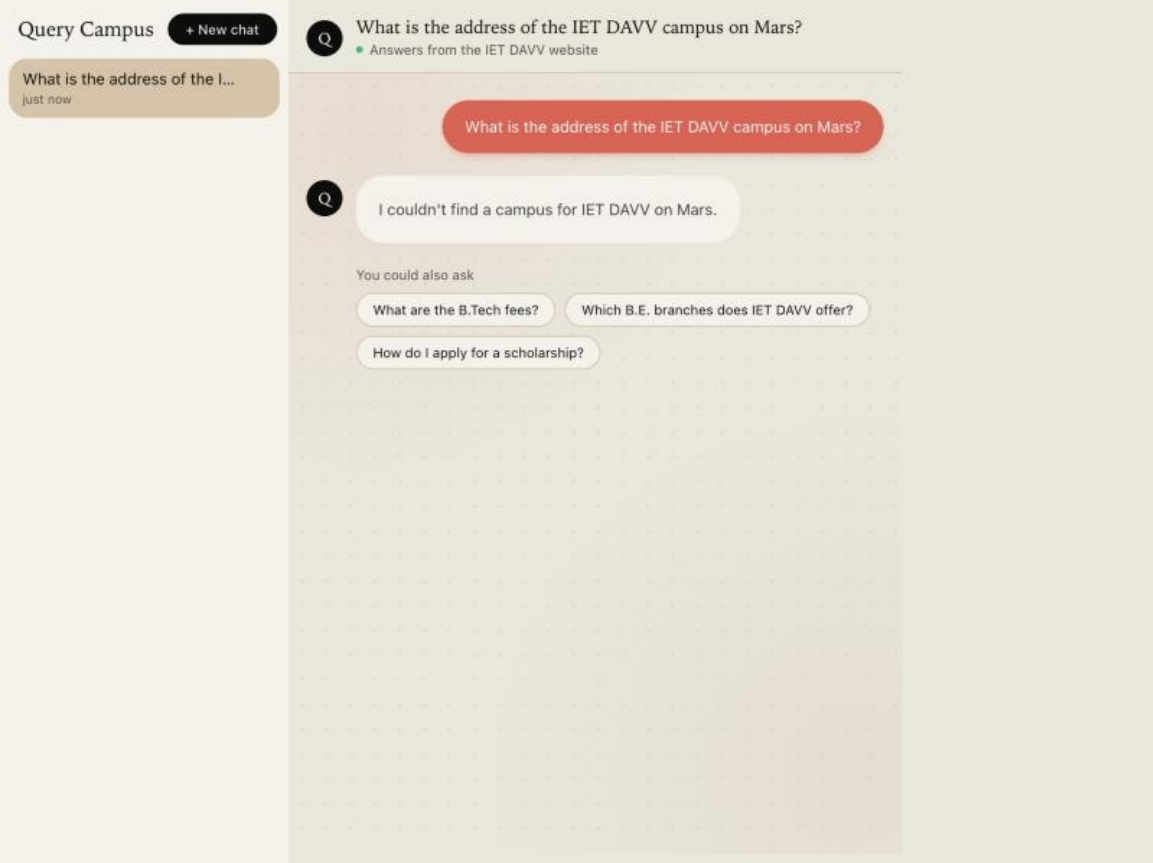


Fig. 4.7: Reply when the information is not available

Test 05: Test case for an unanswerable question

Input: What is the address of the IET DAVV campus on Mars?

System output: I couldn't find a campus for IET DAVV on Mars. (no sources shown)

Result: Passed

Test 06: Test case for a question that has no exact answer on the site

Input: How do I check the exam timetable?

System output: I couldn't find the exact exam timetable on the college website. However, you can regularly visit the official website for updates on the exam schedule, class timetable, academic calendar, syllabus and exam registration.

Result: Passed

Route validation tests. The following tests were run on the REST routes with curl and in the automatic end-to-end tests.

Test 07: Test case for a blank question

Input: POST /query with {“question”: “ ”}

System output: HTTP 400, “question is required”

Result: Passed

Test 08: Test case for a missing question

Input: POST /query with {}

System output: HTTP 400, “question is required”

Result: Passed

Test 09: Test case for a very long question

Input: POST /query with a question of 501 characters

System output: HTTP 400, “question is too long”

Result: Passed

Test 10: Test case for downloading a non-college document

Input: GET /documents/download?url=https://example.com/a.pdf

System output: HTTP 400, “Only PDF documents from the college website can be downloaded”

Result: Passed

Test 11: Test case for an invalid download link

Input: GET /documents/download?url=notaurl

System output: HTTP 400, “url must be a valid URL”

Result: Passed

Test 12: Test case for downloading a real college PDF

Input: GET /documents/download?url=https://www.ietdavv.edu.in/images/timetable/IIyr_IT_A.pdf

System output: HTTP 200, content type application/pdf, a PDF file of about 200 KB

Result: Passed

Test 13: Test case for the visitor limit (limit set to 5 per minute)

Input: Six questions from one address within a minute

System output: The first five get a normal reply; the sixth gets HTTP 429 with a Retry-After header and the message “Too many questions. Please wait a minute and try again.”

Result: Passed

Test 14: Test case for the allowed website address (CORS)

Input: A request from https://evil.example, and one from the address set in CORS_ORIGIN

System output: The allowed address gets the Access-Control-Allow-Origin header; the other address does not

Result: Passed

Test 15: Test case for a server that is not running

Input: Ask a question while the backend is stopped

System output: An error bubble: “Couldn’t reach the server. Please check your connection and try again in a moment.”

Result: Passed

4.3.3 Accuracy test results

Table 4.7 lists the 18 test questions. A case passes when the expected page was found, the required facts are in the answer, and attachments behave as expected. **All 18 cases passed.** The expected page was found in all 14 cases that name a page to find, and it was the first result in 8 of them. Normal answer time was 1.5 to 4 seconds. When Gemini was busy, a backup model answered and a few answers took 20 to 36 seconds, which is still inside the 40-second limit of the system.

Table 4.7: Accuracy test of 18 questions (run of 23 October 2026)

No.	Question	Result	Rank of expected page	Answer time / note
1	Who is Rohit Pathak?	Pass	4	3.8 s
2	What does Rohit Pathak teach?	Pass	1	1.5 s

No.	Question	Result	Rank of expected page	Answer time / note
3	Which classes does Vedpriya Dongre teach?	Pass	1	6.9 s
4	Who is the class coordinator of 4th year CSE section B?	Pass	1	3.3 s
5	What is the research interest of Dr. Sufia Aziz?	Pass	2	1.9 s
6	Who is the HOD of Applied Science?	Pass	1	1.9 s
7	Tell me about Hemant Makwana	Pass	2	2.7 s
8	Who is Ashesh Tiwari?	Pass	8	4.7 s
9	Who is Vijay Kumar Karma?	Pass	2	3.9 s
10	Who is Ravi Sindal?	Pass	2	2.3 s
11	What is Lalit Gehlod's designation?	Pass	1	36.2 s (model busy, backup used)
12	What are the B.Tech fees?	Pass	1	5.2 s
13	Which B.E. branches does IET DAVV offer?	Pass	n/a	2.8 s
14	Send me the 4th year CSE section B timetable	Pass	1	4.0 s
15	Send me the 2nd year IT timetable	Pass	n/a	3.1 s
16	What is the address of the IET DAVV campus on Mars?	Pass	n/a	1.5 s
17	How do I check the exam timetable?	Pass	n/a	20.2 s (model busy, backup used)
18	Where can I see the exam results?	Pass	1	4.9 s

4.3.4 Improvements made after testing

Testing showed several things to improve. Each one was fixed and the tests were run again.

- **More pages in the index.** The first crawl stopped at 600 pages. The page limit was raised and the crawl was repeated. It found 202 more documents, such as faculty biodata PDFs. The index grew from 2,247 to 2,864 chunks.
- **An empty timetable entry** was found in the index. It was removed and the scraper now skips empty entries.
- **Better keyword search.** Different spellings (“exam time table” and “exam timetable”) now match, words from the page address count, and questions like “2nd year IT” also search for “II Year Information Technology”.
- **Better mix of results.** One long page can fill at most three of the 16 result slots, and faculty pages come first for “Who is ...?” questions.
- **Better answers.** When the exact thing is not found, the assistant now says so clearly and mentions related information, and it does not use words such as “excerpts”.
- **Works when one search is down.** If the embedding service is busy, the keyword search alone answers; the index also starts in half a second from a copy on disk instead of waiting 15 to 20 seconds.
- **Faster updates.** The incremental mode embeds only new pages. The 618 new chunks were added without touching the 2,246 old ones.
- **Safe for the public.** Visitor limit, allowed website addresses, maximum question length and a 40-second limit for answers were added.

Chapter-5 Conclusion

What we learned. Building Query Campus taught us how a complete RAG system is put together and where it can fail. We learned to get clean text from messy web pages and PDFs, to choose the chunk size and overlap, and to work with embeddings and a vector database. The most important lesson was that good search matters more than the AI model. Our first version used only search by meaning, and it missed names and numbers. Adding BM25 keyword search and rank fusion fixed this. We also learned to handle slow or limited online services with retries, backup models and time limits, to stream answers with server-sent events, and to test with a fixed set of questions instead of judging by feeling. On the engineering side, we gained experience with NestJS modules, TypeScript, a React 19 chat screen, Tailwind CSS theming, pnpm workspaces and UML diagrams.

What we achieved. The project gives a working assistant for IET DAVV:

- A knowledge base of 1,009 documents (160 website pages, 606 PDFs and 243 timetable documents) in 2,864 searchable chunks.
- Search by meaning plus keywords, answers with citations, and a clear “I couldn’t find it” when the answer is not available, all checked on the running app on 22 and 23 October 2026.
- Answers that start appearing in about one to four seconds, a backup model when one is slow or limited, and a keyword-only mode when the embedding service is busy.
- A simple, light chat screen with saved chats, source links and document cards, and a safe PDF download route.
- Updating by two commands (scrape and ingest) with an incremental mode that adds only new pages, one more command for timetables, and an accuracy test in which all 18 cases pass.

Limitations.

- **No chat memory.** Each question is answered on its own. A follow-up like “and his email?” is not understood.
- **Data can be old.** The data is only as new as the last crawl. The incremental mode skips pages that were already indexed, so a page that was edited needs a full re-index. The

system cannot know if the website itself is out of date or empty (for example, the college's own exam timetable page is empty at the moment).

- **Scanned PDFs are not read.** PDFs that are only pictures have no text, and we do not use OCR.
- **Free plan limits.** The free Voyage AI plan allows only 3 requests a minute for all visitors together. When the limit is reached, the system uses keyword search only, which is a little less exact. A paid plan removes this limit. Indexing is also slow on the free plan (about 12 chunks a minute).
- **English only, no login, no feedback.** Everyone gets the same answers, and users cannot rate answers.
- **Depends on the website layout.** The crawler picks the main content by the layout of the current website. A redesign needs a small code change.
- **Few automatic tests.** The 23 automatic tests cover the helper functions, the chunker and the input checks. Answer quality is checked by the accuracy test and by hand.

Future work.

- Add chat memory so that follow-up questions work.
- Use a paid embedding plan for faster indexing and no limit, and add a re-ranking step and filters by session or department to make search even better.
- Crawl automatically and re-index only the pages that changed.
- Add OCR for scanned notices, and support for Hindi.
- Add thumbs up/down buttons to collect feedback and grow the test set.
- Add a WhatsApp or Telegram bot, an admin page for crawl status, and automatic testing on every code change.
- Re-use the system for other colleges by changing the website address and content selector.

Conclusion. Query Campus shows that a small, low-cost and self-hosted RAG system can turn a large and hard-to-use college website into a simple chat service. Every answer is based on the college's own pages, shows its sources, and says "I don't know" when the answer is not there. This makes it more reliable than manual browsing or a general chatbot.

REFERENCES

The references follow the IEEE style. All references are websites and documentation pages (accessed in September–October 2026).

[1] Perplexity AI, “Perplexity – AI-Powered Conversational Search Engine,” 2024. [Online]. Available: https://www.perplexity.ai

[2] ChatPDF, “ChatPDF – Chat with Any PDF Using AI,” 2024. [Online]. Available: https://www.chatpdf.com

[3] Ivy.ai, “Generative AI Chatbots for Higher Education,” 2024. [Online]. Available: https://ivy.ai/higher-education

[4] Ocelot, “AI Chatbot for Higher Education Student Engagement,” 2024. [Online]. Available: https://ocelotbot.com/features/chatbot

[5] Element451, “BoltBot – Intelligent AI Chatbot for Higher Education,” 2024. [Online]. Available: https://element451.com/boltbot-intelligent-chatbot

[6] Gravyty, “AI Chatbot for Higher Education,” 2024. [Online]. Available: https://gravyty.com/products/ai-chatbot-for-higher-education

[7] Pinecone Systems Inc., “Pinecone Vector Database Documentation.” [Online]. Available: https://docs.pinecone.io

[8] Voyage AI, “Embedding Models Documentation.” [Online]. Available: https://docs.voyageai.com

[9] Google, “Gemini API Developer Documentation.” [Online]. Available: https://ai.google.dev/gemini-api/docs

[10] NestJS, “NestJS Documentation.” [Online]. Available: https://docs.nestjs.com

[11] Meta Open Source, “React Documentation.” [Online]. Available: https://react.dev

[12] Axios, “Axios Documentation.” [Online]. Available: https://axios-http.com/docs

[13] Cheerio, “Cheerio Documentation.” [Online]. Available: https://cheerio.js.org

[14] L. Ongaro, “MiniSearch Documentation.” [Online]. Available: https://lucaong.github.io/minisearch

[15] Institute of Engineering & Technology, DAVV, Indore, “Official website.” [Online]. Available: https://www.ietdavv.edu.in

[16] IET DAVV, “Class timetable portal.” [Online]. Available: https://timetable.ietonline.in

APPENDIX

A. Abbreviation List

Abbreviation	Meaning
API	Application Programming Interface
BFS	Breadth-First Search
BM25	Best Matching 25 (probabilistic ranking function)
CORS	Cross-Origin Resource Sharing
CSE / IT	Computer Science Engineering / Information Technology
DAVV	Devi Ahilya Vishwavidyalaya
HTML	HyperText Markup Language
HTTP(S)	HyperText Transfer Protocol (Secure)
IET	Institute of Engineering & Technology
JSON	JavaScript Object Notation
LLM	Large Language Model
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RRF	Reciprocal Rank Fusion
SPA	Single-Page Application
SRS	Software Requirements Specification
SSE	Server-Sent Events
UI / UML	User Interface / Unified Modeling Language
URL	Uniform Resource Locator

B. Glossary

Term	Meaning
Chunk	A passage of up to 1,500 characters cut from a document, the unit that is embedded and retrieved.
Embedding	A list of numbers (here 1,024) representing the meaning of a text; similar texts have similar embeddings.
Grounding	Requiring the model to answer only from supplied evidence and to cite it.
Hallucination	A fluent but unsupported or false statement produced by a language model.
Index (vector)	A database structure that finds the vectors nearest to a query vector quickly.
Ingestion	The process of chunking, embedding and storing documents.
Session	An academic term such as 2026-2027 (Jul–Dec 2026) used by the timetable portal.
Stale-while-revalidate	Serving the existing index while a new one is built in the background.

C. User Manual

C.1 For students, applicants and staff

1. Open the Query Campus web address in any modern browser.
2. Type a question in plain English in the box at the bottom (or click one of the suggestion cards) and press Enter or the arrow button.
3. Read the answer as it appears. Facts that came from the college sources are listed under “Sources”; click a link to open the original page.
4. To get a document, ask for it explicitly, for example “Send me the syllabus of 3rd semester”. A document card appears; use Open to view it or Download to save a college PDF.
5. Use “+ New chat” to start a new conversation, select an earlier chat in the sidebar to continue it, or click the cross beside a chat to delete it. Chats are kept only in your browser.

6. If the assistant says it does not have the information, check the official website or contact the college.

C.2 For the operator

Prerequisites: Node.js, pnpm and API keys for Gemini, Voyage AI and Pinecone.

1. Install dependencies from the repository root with “pnpm install”.
2. Copy apps/backend/.env.example to apps/backend/.env and fill GEMINI_API_KEY, VOYAGE_API_KEY and PINECONE_API_KEY; copy apps/frontend/.env.example to apps/frontend/.env.
3. From apps/backend run “pnpm run scrape” to crawl the website, then “pnpm run ingest” to index it. Re-run both when the website changes.
4. Run “pnpm run timetables” at the start of every semester to refresh class and faculty timetables.
5. Start the services with “pnpm run dev:backend” (port 3001) and “pnpm run dev:frontend” (port 5173).
6. After changing the prompt, model or retrieval settings run “pnpm run eval” (add --retrieval-only to avoid using Gemini quota) and check that there are no regressions.

D. Configuration Variables

Variable	Default	Meaning
PORT	3001	Backend port
CORS_ORIGIN	(empty = any)	Website addresses allowed to call the API (comma separated); set it in production
RATE_LIMIT_PER_MINUTE	20	Questions allowed per visitor per minute (0 = no limit)
TRUST_PROXY	0	Set to 1 behind one proxy so the real visitor address is used
EMBED_BATCH_SIZE / EMBED_BATCH_DELAY_MS	5 / 21000	Indexing pace; use e.g. 64 / 0 on a paid Voyage plan

Variable	Default	Meaning
SCRAPE_BASE_URL	https:// www.ietdavv.edu.in/	Website to crawl
SCRAPE_MAX_PAGES	600	Maximum pages visited
GEMINI_API_KEY / GEMINI_MODEL	(secret) / gemini-3.5-flash-lite	Generation key and primary model
GEMINI_FALLBACK_MODELS	gemini-3.1-flash-lite, gemini-3.5-flash	Models tried when the primary fails
GEMINI_THINKING_LEVEL	minimal	Lower thinking gives faster answers
VOYAGE_API_KEY / VOYAGE_MODEL	(secret) / voyage-3	Embedding key and model
VOYAGE_DIMENSION	1024	Vector size; must match the index
PINECONE_API_KEY / PINECONE_INDEX	(secret) / query-campus	Vector database key and index name
VITE_API_URL (frontend)	http://localhost:3001	Backend address used by the browser

E. REST Interface Summary

Method	Path	Description
POST	/query	Body {question} (up to 500 characters). Returns {answer, sources, attachments} as JSON. HTTP 429 after 20 questions a minute from one address.
POST	/query/stream	Same input; returns server-sent events: delta {text}, done {answer, sources, attachments}, error {message}.
GET	/documents/download?url=	Streams a PDF from the college website as a download.
GET	/health	Returns {status: ok}.

F. Deployment

The backend and the frontend are deployed separately (details are in the README of the project).

1. Backend on any Node host (for example Render or Railway): run “pnpm install”, “pnpm --filter backend build” and “pnpm --filter backend start:prod”.
2. Set the environment variables of the backend: the three API keys, CORS_ORIGIN (the website address), RATE_LIMIT_PER_MINUTE and TRUST_PROXY=1 when the host uses a proxy.
3. The Pinecone index must be filled before the first visitor comes: run the scrape and ingest commands once from your computer.
4. Frontend on any static host (for example Netlify or Vercel): build with VITE_API_URL set to the backend address, then upload apps/frontend/dist.
5. Open GET /health on the backend, then ask a few questions on the website to check that everything works.